METODOLOXÍAS DE DESENVOLVEMENTO

Metodologías Ágiles: Kanban

Anxo Pérez & Javier Parapar

anxo.pvila@udc.es

Information Retrieval Lab
Computer Science Department
University of A Coruña

Contexto

Prescriptivo vs Adaptativo

RUP (120+)

- Architecture Reviewer
- Business Designer
- Business-Model Reviewer
- Business-Process Analyst
- Capsule Designer
- Change Control Manager
- Code Reviewer
- Configuration Manager
- Course Developer
- Database Designer
- Deployment Manager
- Design Reviewer
- Designer
- Graphic Artist
- Implementer
- Integrator
- Process Engineer
- Project Manager
- Project Reviewer
- Requirements Reviewer
- Requirements Specifier
- Software Architect
- Stakeholder
- System Administrator
- System Analyst
- Technical Writer
- Test Analyst
- Test Designer
- Test Manager
- Tester
- Tool Specialist
- User-Interface Designer
- Architectural analysis
- Assess Viability of architectural proof-of-concept
- Capsule design
- Class design
- Construct architectural proof-of-concept
- Database design
- Describe distribution
- Describe the run-time architecture
- Design test packages and classes
- Develop design guidelines
- Develop programming guidelines
- Identify design elements
- Identify design mechanisms
- Incorporate design elements
- Prioritize use cases
- Review the architecture
- Review the design
- Structure the implementation model
- Subsystem design
- Use-case analysis
- Use-case design
- Analysis model
- Architectural proof-of-concept
- Bill of materials
- Business architecture document
- Business case
- Business glossary
- Business modeling guidelines
- Business object model
- Business rules
- Business use case
- Business use case realization
- Business use-case model
- Business vision
- Change request
- Configuration audit findings
- Configuration management plan
- Data model
- Deployment model
- Deployment plan
- Design guidelines
- Design model
- Development case
- Development-organization assessment
- End-user support mateirla
- Glossary
- Implementation model
- Installation artifacts
- Integration build plan
- Issues list
- Iteration assessment
- Iteration plan
- Manual styleguide
- Programming guidelines
- Quality assurance plan
- Reference architecture
- Release notes
- Requirements attributes
- Requirements management plan
- Review record
- Risk list
- Risk management plan
- Software architecture document
- Software development plan
- Software requirements specification
- Stakeholder requests
- Status assessment
- Supplementary business specification
- Supplementary specification
- Target organization assessment
- Test automation architecture
- Test cases
- Test environment configuration
- Test evaluation summary
- Test guidelines
- Test ideas list
- Test interface specification
- Test plan
- Test suite
- Tool guidelines
- Training materials
- Use case model
- Use case package
- Use-case modeling guidelines
- Use-case realization
- Use-case storyboard
- User-interface guidelines
- User-interface prototype
- Vision
- Work order
- Workload analysis model

XP (13)

- Whole team
- Coding standard
- TDD
- Collective ownership
- Customer tests
- Pair programming
- Refactoring
- Planning game
- Continuous integration
- Simple design
- Sustainable pace
- Metaphor
- Small releases

Scrum (9)

- Scrum Master
- Product Owner
- Team
- Sprint planning meeting
- Daily Scrum
- Sprint review
- Product backlogt
- Sprint backlog
- BUrndown chart

Kanban (3)

- Visualize the workflow
- Limit WIP
- Measure and optimize lead time

←Prescripción ——————————— Adaptación →

Kanban

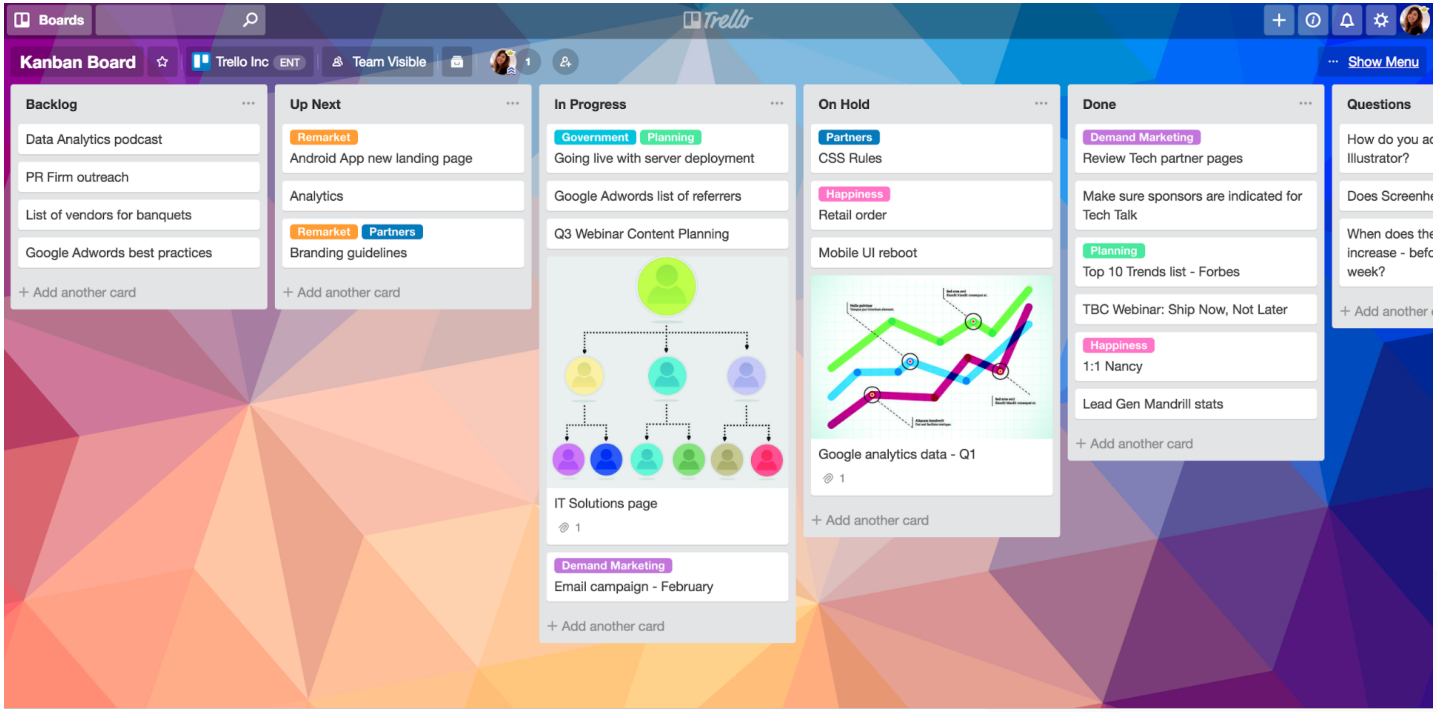
> *Kanban establishes a new **balance** on the relationship between people and processes, establishing a way to empower people to **design** their **own processes**. Hence, Kanban is a collaborative exercise for process design and offers thinking and action tools to empower people to **evolve processes** by themselves*
>
> *Alisson Vale*

看板

Introducción a Kanban

- El término **Kanban** deriva del japonés: ***kan*** significa **visual** y ***ban*** significa **tarjeta**.
- Es una metodología de organización del trabajo basada en **señales visuales** para gestionar la carga de trabajo del equipo de desarrollo.
- A través del uso de tableros y tarjetas, el sistema **Kanban** permite identificar **retrasos**, mejorar los **tiempos** de servicio de tareas y optimizar la **calidad** del proceso.
- Kanban es especialmente efectivo en equipos con alta **madurez** y **autodisciplina**, y puede incrementar significativamente la capacidad del equipo para responder a cambios.

Introducción a Kanban



¿Es Kanban agile?

- Depende... **Kanban** puede considerarse compatible con el desarrollo ágil, aunque TÉCNICAMENTE no es una metodología agile en el sentido estricto de metodologías formalizadas como Scrum o XP.
- Más bien, **Kanban** es un sistema visual y un enfoque de gestión de flujo de trabajo basado en principios de mejora continua, flexibilidad, y transparencia, aspectos que se alinean muy bien con los valores del Manifiesto Ágil.

¿Es Kanban agile?

- Aspectos que lo acercan a lo ágil:
  - **Enfoque en la entrega continua**: Kanban permite adaptarse a los cambios al optimizar el flujo de trabajo, manteniendo siempre visible el progreso en cada tarea.
  - **Flexibilidad y mejora continua**: Permite introducir cambios en cualquier momento, sin la necesidad de esperar a una iteración específica, lo que lo hace adaptable a cambios constantes y evolutivos.
  - **Colaboración y transparencia**: Al hacer visible el trabajo en una pizarra (física o digital), fomenta la colaboración y transparencia, principios fundamentales en el desarrollo ágil.

¿Es Kanban agile?

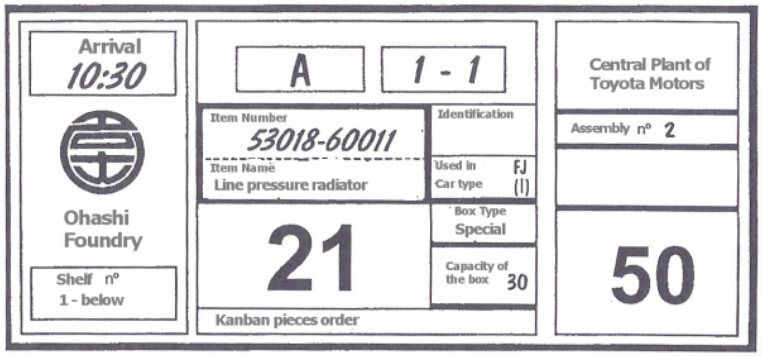


Origen de Kanban en Toyota

- **Década de 1940:** Toyota enfrenta dificultades para competir con la producción en masa estadounidense debido a recursos limitados tras la Segunda Guerra Mundial.
- Inspirados en los supermercados americanos, Toyota desarrolla un sistema de **producción justo a tiempo (JIT)**, produciendo solo lo necesario en el momento preciso.
- Taiichi Ohno, ingeniero industrial de Toyota, introduce las **tarjetas Kanban** para comunicar la necesidad de producción en cada fase de la cadena.

Cómo Funcionaba Kanban en Toyota

- En el sistema Kanban de Toyota, cada **etapa** del proceso de producción usaba tarjetas que servían como **señales visuales** para gestionar el flujo de trabajo.
- Una tarjeta Kanban indicaba que una fase posterior necesitaba piezas, y solo entonces se iniciaba la producción.
- **Objetivo:** evitar la sobreproducción y el exceso de inventario, garantizando un flujo continuo sin interrupciones.



Principios Fundamentales del Kanban de Toyota

- **Just in Time (JIT):** Producir solo lo necesario en el momento preciso, evitando exceso de inventario.
- **Producción en base a demanda:** Cada fase de la cadena produce en función de la señal visual (tarjeta) que indica una necesidad.
- **Mejora continua:** Toyota aplicaba un proceso de mejora constante (Kaizen), optimizando el flujo de trabajo y eliminando ineficiencias.

Kanban Evoluciona al Desarrollo Ágil

- En los 2000s, los principios de Kanban fueron adoptados y adaptados por la **gestión ágil de proyectos**.
- La visualización del flujo de trabajo y el límite de trabajo en progreso (WIP) se convirtieron en principios clave para gestionar proyectos de software de forma eficiente.
- Hoy en día, Kanban permite a equipos de desarrollo reducir **cuellos de botella**, mejorar el flujo de trabajo y adaptarse rápidamente a los cambios en los requisitos.

Kanban: Concepto General

- Kanban es un sistema de trabajo **just in time**:
  - Evita **stock innecesario**, limitando la inversión y el almacenamiento de elementos que aún no son necesarios.
  - Ayuda a **priorizar el trabajo**, enfocándose en tareas esenciales y evitando la sobrecarga de tareas no prioritarias.
  - **Protege la carga de trabajo del equipo**, asegurando que las personas no se sobrecarguen y puedan mantener una calidad constante.
- Kanban introduce cambios progresivos en el ciclo de vida del desarrollo, adaptando el flujo de trabajo sin necesidad de empezar de cero.
- Un concepto clave de Kanban es limitar el **trabajo en curso** o **Work In Progress (WIP)**, manteniendo el enfoque y la calidad del trabajo.

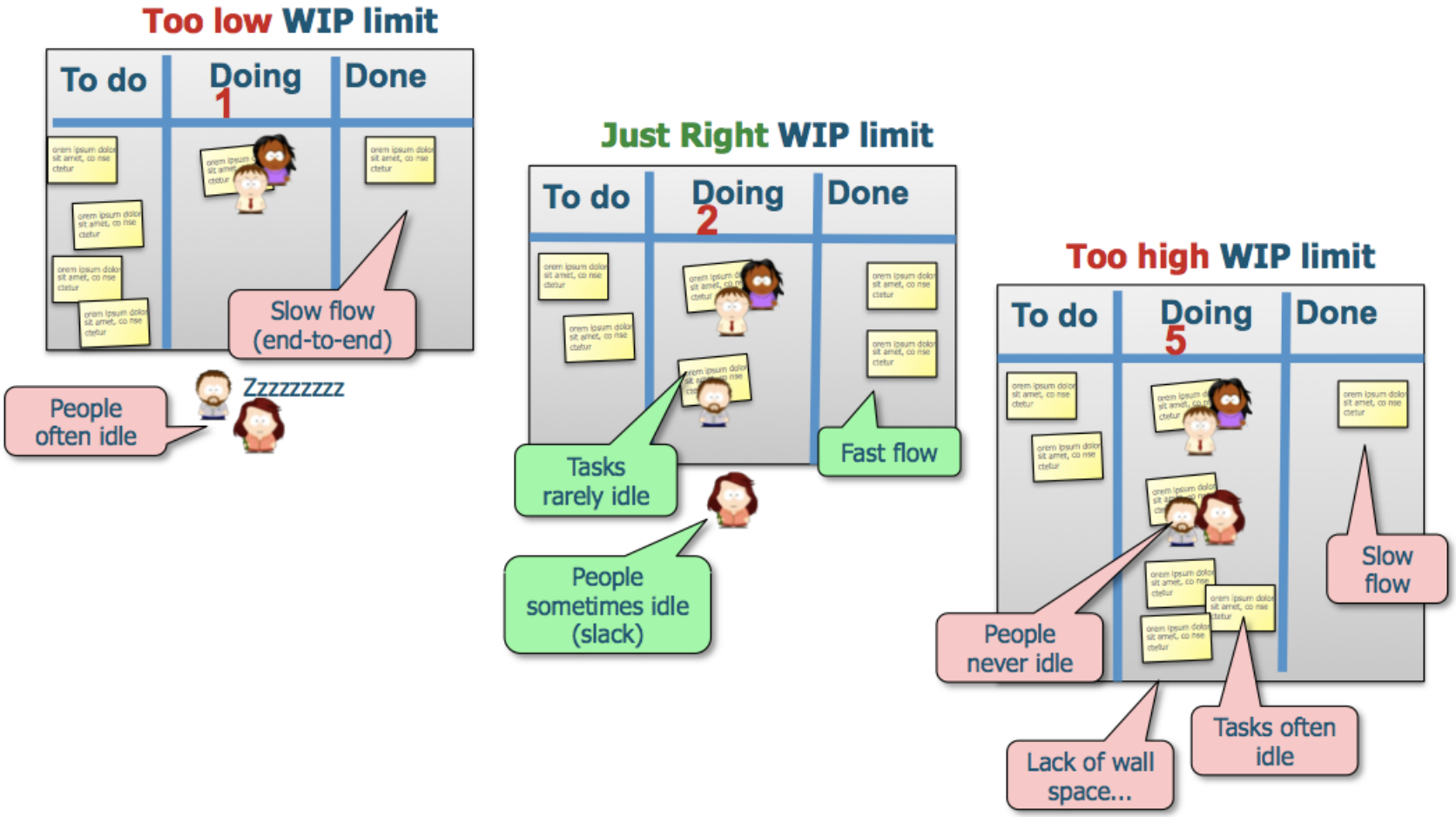
Concepto de WIP (Work In Progress)

- **Work In Progress (WIP):** Cantidad de tareas o trabajos activos en un momento dado, en cada fase del proceso.
- Limitar el WIP permite a los equipos concentrarse en menos tareas a la vez, reduciendo errores y mejorando la calidad.
- Está demostrado que, a medida que el WIP aumenta, se disminuye la capacidad de atención del equipo y se incrementa la probabilidad de errores.

Beneficios de limitar el WIP

- **Mejora de la calidad:** Aumenta el nivel de concentración en cada tarea y reduce la tasa de errores.
- **Reducción de tiempo de ciclo:** Las tareas avanzan de forma más rápida desde su inicio hasta su finalización.
- **Mayor enfoque y satisfacción del equipo:** Al reducir la multitarea, los equipos pueden trabajar de manera más fluida y eficaz.

WIP: Trabajos por estado de flujo



WIP

- Limitar el WIP no implica reducirlo a niveles demasiado bajos; el objetivo es encontrar el **punto óptimo** que permita la máxima eficiencia sin sobrecargar al equipo.
- **Equilibrio en el WIP:**
  - Si el WIP es demasiado bajo, puede reducir la capacidad de producción y ralentizar el flujo de trabajo, causando menos entregas a producción.
  - Si el WIP es demasiado alto, se generan cuellos de botella y aumentan los errores debido a la sobrecarga de tareas.

WIP

- **Objetivo:** Establecer un WIP adecuado que mantenga al equipo ocupado y productivo, sin comprometer la calidad ni el tiempo de entrega.
- El equilibrio ideal del WIP asegura un flujo de trabajo estable y continuo, permitiendo al equipo **entregar valor de forma constante** y **aumentar la predictibilidad** en las entregas.

Ejes de Kanban

- Kanban se basa en una serie de **principios** y **ejes** fundamentales que guían su enfoque para la gestión del trabajo, centrando la atención en la eficiencia y la mejora continua.
- Cada eje contribuye a crear un sistema de trabajo más **organizado** y **ágil**, que facilita la entrega de valor constante y el cumplimiento de los objetivos del proyecto.

Ejes de Kanban

- Los cuatro ejes principales que veremos son:
  - **Calidad garantizada:** Focalización en hacer las cosas bien desde el inicio.
  - **Reducción del desperdicio:** Enfocado en eliminar lo innecesario y maximizar la eficiencia.
  - **Mejora continua:** Una cultura de adaptación y optimización constante.
  - **Flexibilidad:** Capacidad para adaptarse a las necesidades cambiantes del entorno.
- Estos pilares hacen de Kanban una herramienta potente para la gestión de proyectos, mejorando tanto la **productividad** del equipo como la **satisfacción** del cliente.

Ejes: Calidad garantizada

- En Kanban, la calidad es prioritaria: todo lo que se hace debe salir **bien a la primera**, minimizando errores y retrabajos.
- No se premia la **rapidez**, sino la **calidad** del producto final.
- Arreglar errores después de completar una tarea puede resultar más costoso que hacerlo bien desde el inicio.

Klaus Leopold: "It's nonsense to blame testers for the quality of the software. Quality starts at the board of directors."

Ejes: Calidad garantizada

- Buenas prácticas de calidad garantizada:
  - Cumplir con coding standard guidelines
  - Involucración del cliente
  - Automated testing
  - DevOps y CI/CD pipeline

Ejes: Reducción del desperdicio

- Kanban se basa en el principio de hacer **solamente lo necesario** y hacerlo bien.
- Esto implica una reducción de todo aquello que no aporta valor directo, eliminando lo **superficial** o innecesario.

> *"YAGNI originally is an acronym that stands for "You Aren't Gonna Need It". It is a mantra from XP that's often used generally in agile software teams. It's a statement that some capability we presume our software needs in the future should not be built now because You Aren't Gonna Need It"*
>
> *Martin Fowler*

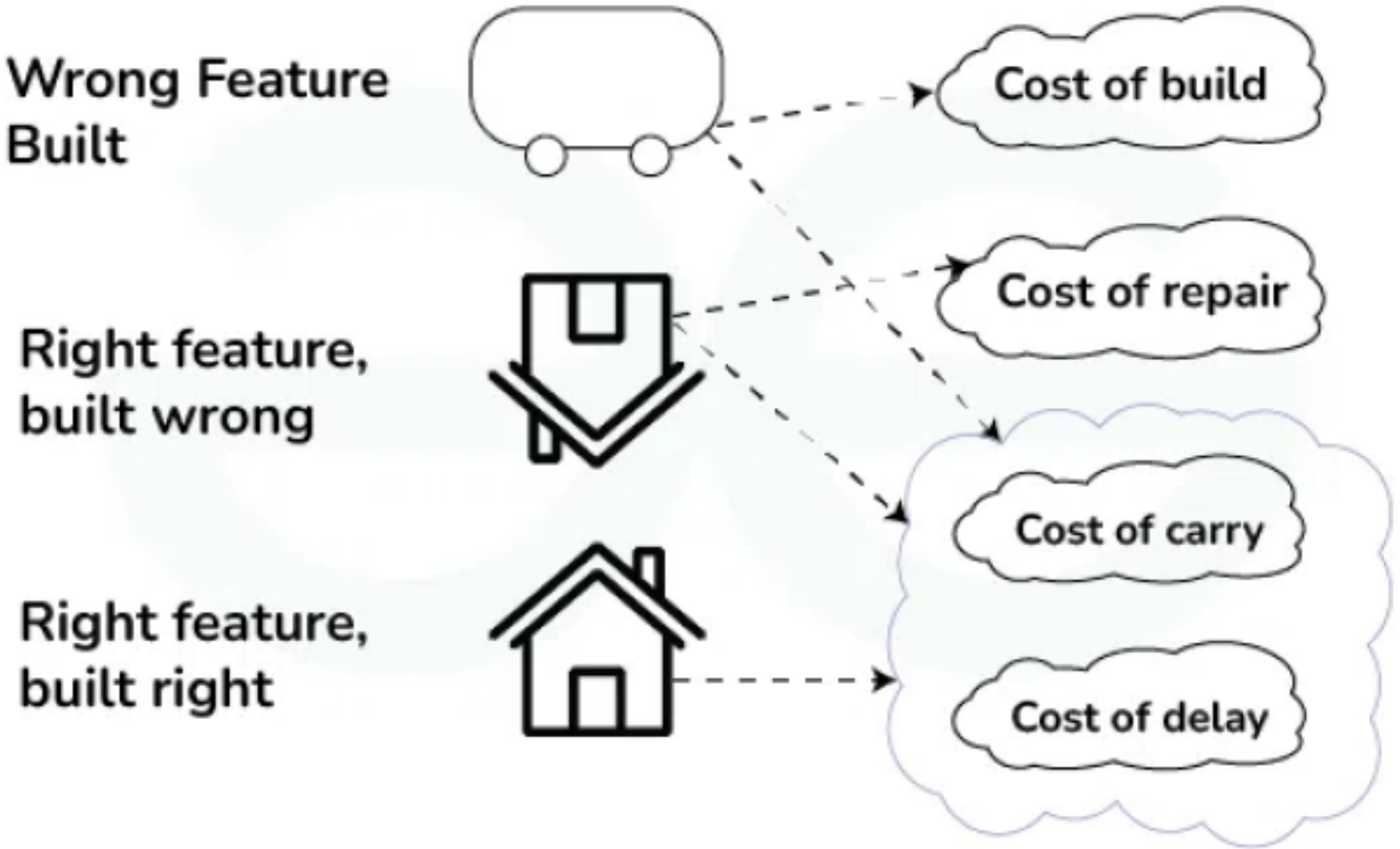
¿Qué es YAGNI?

- **YAGNI** (*You Aren't Gonna Need It*) es un principio ágil que sugiere que no debemos construir una funcionalidad a menos que sea realmente necesaria.
- Este principio promueve el enfoque en desarrollar solo lo que es necesario en el momento actual, evitando la tentación de anticipar necesidades futuras.
- Al aplicar YAGNI, los equipos de desarrollo pueden reducir la complejidad del código, facilitar el mantenimiento y entregar valor rápidamente.
- **Ejemplos comunes de no aplicar YAGNI:**
  - Construir funcionalidades adicionales que aún no han sido solicitadas.
  - Añadir opciones de configuración innecesarias en una fase temprana.

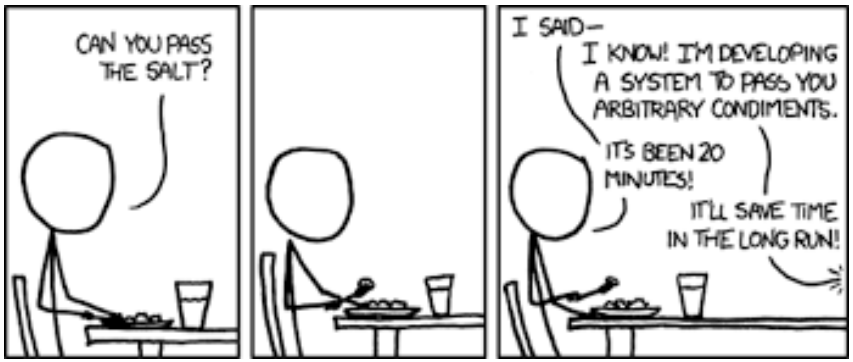
¿Por qué cumplir con YAGNI?

- **Cost of build:** Todos los recursos invertidos en analizar, desarrollar y testear una feature
- **Cost of carry:** La complejidad añadida por la feature, hace el software más difícil de modificar/refactorizar
- **Cost of delay:** Esos recursos pueden ser utilizados en construir y entregar features que SÍ son necesarias
- **Coste de reparación:** El esfuerzo invertido en modificar una feature porque no es lo que realmente se necesitaba, junto con el largo plazo que supondrá una feature más a mantener

¿Qué es YAGNI?


Wrong Feature Built
Right feature, built wrong
Right feature, built right
Cost of build
Cost of repair
Cost of carry
Cost of delay

¿Qué es YAGNI?


CAN YOU PASS THE SALT?
I SAID—
I KNOW! I'M DEVELOPING A SYSTEM TO PASS YOU ARBITRARY CONDIMENTS.
IT'S BEEN 20 MINUTES!
IT'LL SAVE TIME IN THE LONG RUN!

Errores Comunes al No Aplicar YAGNI

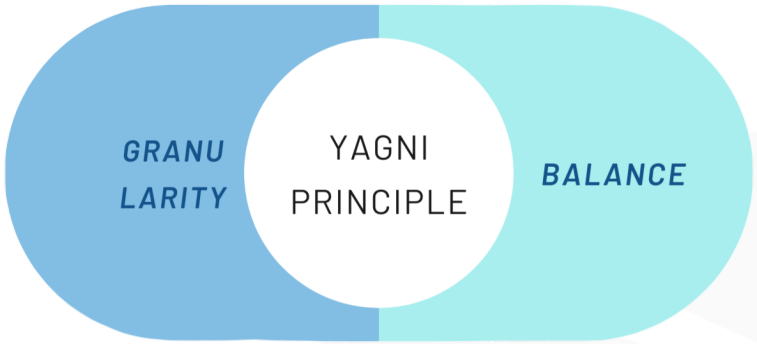
- **Módulos de Autenticación Complejos**
  - Desarrollar un sistema de autenticación avanzado con múltiples factores antes de que la aplicación requiera algo más que un inicio de sesión básico.
  - Resultado: Complejidad innecesaria y mayores costos de mantenimiento.
- **APIs Generalizadas**
  - Crear una API con múltiples puntos de extensión y configuraciones antes de que los usuarios las necesiten.
  - Resultado: Dificultad de uso y pruebas sin una demanda real.

Errores Comunes al No Aplicar YAGNI

- **Funcionalidades de Reporte Extensivas**
  - Implementar un sistema de reportes con filtros avanzados sin que los usuarios lo hayan solicitado.
  - Resultado: Horas de trabajo invertidas en funcionalidades que no se utilizan.
- Apegándose a YAGNI, se evita implementar características innecesarias, facilitando un desarrollo más ágil y eficiente.

¡Cuidado al aplicarlo!!

- **Importante**: YAGNI no significa ignorar el futuro, sino tomar decisiones considerando el contexto sin anticipar necesidades que tal vez nunca se materialicen.
- El objetivo final es crear un sistema que sea más **comprensible y manejable**, incluso si eso implica escribir más código en algunas áreas.



Ejes: Mejora continua

- Kanban no es solo un método de gestión, sino un sistema de **mejora continua**.
- El enfoque está en ajustar y mejorar continuamente los proyectos para alcanzar los objetivos.

"Initially it is called continuous irritation, eventually continuous improvement."

Ejes: Flexibilidad

- La flexibilidad de Kanban permite que las tareas se seleccionen de un **backlog** en función de las prioridades actuales.
- Las prioridades de las tareas entrantes se pueden **reevaluar** según las necesidades cambiantes del equipo o del negocio.
- Esta capacidad de ajustar el foco permite al equipo responder rápidamente a situaciones imprevistas.

Principios en Kanban

Principios

En Kanban se consideran 4 principios básicos:

- **Comprensión**: Entiende lo que haces y empieza por entender lo que estás haciendo en este momento
- **Acuerdo**: acuerda perseguir el cambio continuo e incremental
- **Respeto**: respetar funciones existentes, puestos, y responsabilidades en el equipo
- **Liderazgo**: apoyar las acciones de liderazgo en todos los niveles de su organización

Comprensión en Kanban

> *"Insist on understanding because a healthy process that can't defend itself is a sign that you've forgotten what you believe."*
>
> *The Process Myth, Rands in Repose*

- El principio de **Comprensión** se centra en entender a fondo lo que se está haciendo y cómo se realiza el trabajo.
- Esto incluye conocer tanto los **detalles esenciales** de los procesos como el **enfoque general** de la organización frente a los cambios.
- No implica un análisis exhaustivo previo, sino la capacidad de realizar **pequeños cambios controlados** que se puedan evaluar rápidamente.
- Es clave **calibrar cada cambio** según el conocimiento adquirido de la situación y los resultados de estos pequeños experimentos.

Comprensión en Kanban

- ¿Quiénes son tus clientes? ¿Qué les frustra sobre el servicio que reciben en la actualidad?
- ¿Quién más tiene poder de veto sobre lo que se entrega?
- ¿Cuáles son los límites de la organización? ¿Cuáles son las frustraciones de las personas que trabajan dentro del sistema? ¿Y de los de fuera?
- ¿Cómo llega el trabajo? ¿Con qué frecuencia y con qué regularidad?
- ¿Qué tipo de trabajo hay? Los diferentes flujos de trabajo, riesgos diferentes, diferentes expectativas de los clientes?
- ¿Cómo y con qué frecuencia se entrega?
- ¿Cuánto tiempo toma todo el proceso de extremo a extremo?

Value Stream Map y Knowledge Discovery Process

- Para entender el flujo de trabajo, hay dos enfoques principales:
  - **Knowledge Discovery Process:** Organizar los elementos de trabajo actuales según sus estados y el conocimiento adquirido a lo largo del proceso (con el cliente y del propio equipo).
    - Enfatiza la organización del trabajo, no de las personas.
    - Útil para identificar en qué etapa está cada tarea y cómo evoluciona el conocimiento del cliente.

Value Stream Map y Knowledge Discovery Process

- **Value Stream Map (VSM):** Trabajar hacia atrás desde la entrega final, identificando las actividades y sus dependencias.
  - El VSM muestra cada paso del proceso de producción y cómo cada actividad añade valor.
  - Ayuda a identificar cuellos de botella y áreas donde el trabajo se detiene.
- En los dos casos es importante identificar dónde se encuentran los elementos de trabajo y qué desencadena el inicio de las actividades.

Ejemplo de Knowledge Discovery Process

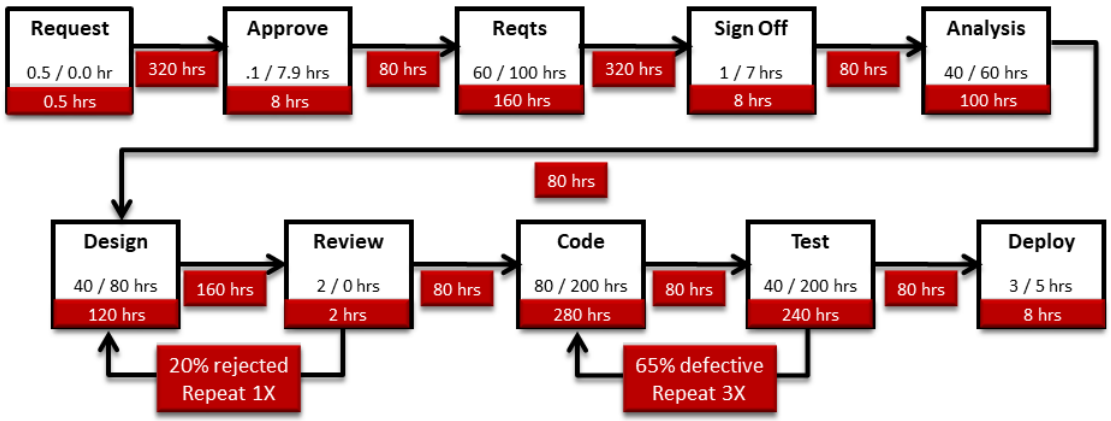
- **Desarrollo de un CRM (Customer Relationship Management):**
  - Las tareas se organizan en estados como *recopilación de requisitos*, *desarrollo de módulos de contacto*, *integración con otras herramientas*, *pruebas de usabilidad*, y *retroalimentación del cliente*.
  - A medida que el equipo avanza por cada estado, adquiere un conocimiento más profundo de las necesidades de los usuarios finales y ajusta el desarrollo para satisfacer mejor estas necesidades.
  - Esto permite un enfoque centrado en el cliente, donde cada iteración se enfoca en comprender mejor y atender sus expectativas.

Value Stream Map (VSM)

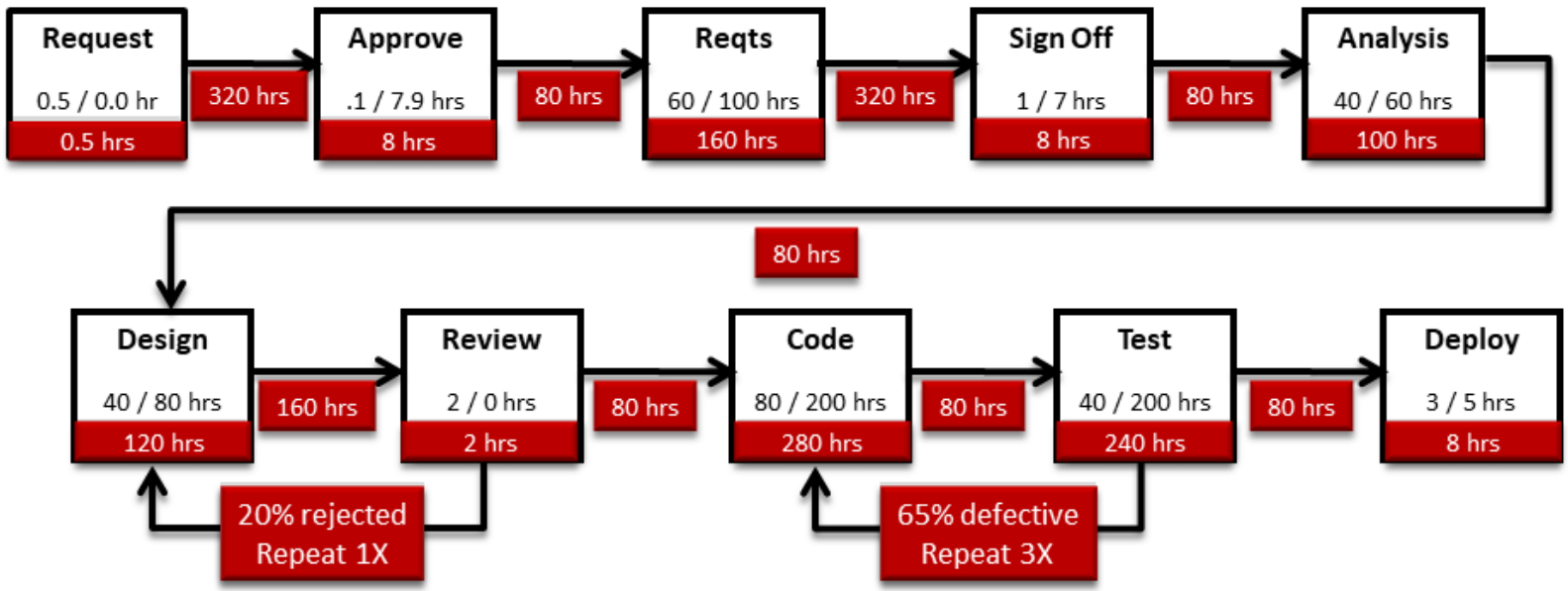
- El Mapa del flujo de valor es una herramienta utilizada en Lean manufacturing y/o metodologías ágiles para analizar los flujos de materiales e información necesarios para poner a disposición del cliente un producto o servicio
  - **Identificar** sobre el mapa los **desperdicios** que se encuentran (aquello que no aporta valor para el cliente): sobreproducción, tiempo de espera, transportes innecesarios, exceso de procesado, inventario, movimientos innecesarios y defectos
  - **Dibujar el mapa de estado futuro**, es decir, el mapa como queda una vez eliminados los desperdicios
  - Implementar un plan de **acciones de mejora** (eventos kaizen) para llegar al mapa de estado futuro

Value Stream Map (VSM)

La definición de la cadena de valor (***value chain***) por la que pasan los ítems de trabajo es importante tanto para analizar el flujo y detectar cuellos de botella, se usan los **value stream map**



Value Stream Map (VSM)



Tiempos en el Value Stream Map

- **Cycle Time (CT)**: Tiempo total desde el inicio hasta el final de un proceso o actividad específica. Incluye tanto el tiempo de trabajo como el tiempo de espera.
- **Lead Time (LT)**: Tiempo total desde que un cliente realiza una solicitud hasta que la recibe. Abarca todos los procesos y tiempos de espera en la cadena.
- **Wait Time (WT)**: Tiempo que un elemento pasa esperando entre diferentes etapas del proceso, como aprobaciones o disponibilidad de recursos.

Tiempos en el Value Stream Map

- **Processing Time (PT)**: Tiempo efectivo de trabajo realizado sobre un elemento, excluyendo esperas y otros retrasos.
- **Throughput** se refiere a la cantidad de ítems de trabajo (como tareas, historias de usuario o tickets) que pasan a través del sistema en un período de tiempo específico. Si un equipo de desarrollo de software completa 15 tickets de soporte y 10 features en una semana, su Throughput semanal sería de 25 ítems.

Creando un Value Stream Map

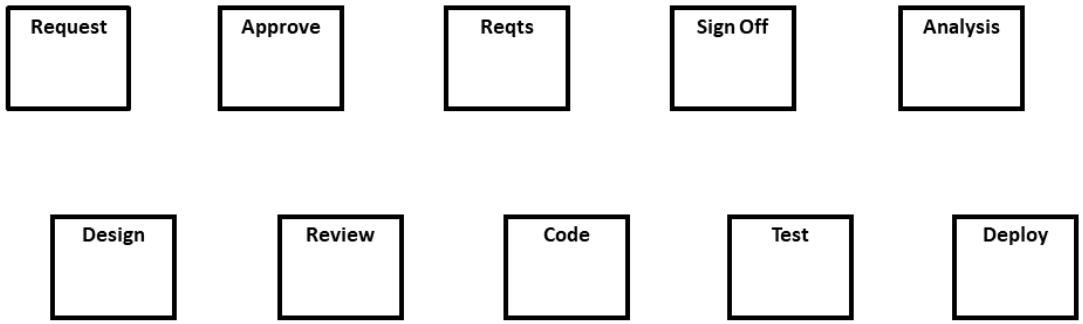
- Antes de comenzar un **Value Stream Map**, es importante recordar que se trata de un mapa sobre cómo se trabaja en un proyecto en particular.
- En otras palabras, estamos mapeando el **flujo de trabajo** para un proyecto específico, no la cantidad de tiempo que las personas dedican a diferentes proyectos.
- No existe una única forma de crear un **Value Stream Map**. Sin embargo, hay un proceso de seis pasos que ha demostrado ser efectivo en muchas ocasiones:

Pasos para Crear un Value Stream Map

1. **Identificar las acciones** que se realizan durante el proyecto.
2. Especificar el **tiempo en el calendario** durante el cual se llevan a cabo estas acciones.
3. Especificar cuánto tiempo de esas acciones fue **trabajo efectivo** y cuánto tiempo se pasó **esperando**.
4. Especificar la cantidad de tiempo entre cada uno de estos pasos.
5. Identificar y señalar cualquier **retroceso** presente en el flujo de trabajo.
6. Sumar el **tiempo total de calendario** del proyecto y calcular qué parte de él fue **trabajo efectivo**.

Paso 1: Identificar las acciones que se realizan

- En vuestra práctica en etapas finales, podríamos tener un proceso en el que las acciones pasan por los siguientes pasos:
  - **Solicitud** → **Aprobación** → **Requisitos** → **Firma** → **Análisis** → **Diseño** → **Revisión** → **Codificación** → **Pruebas** → **Despliegue**.
- Podemos escribir los pasos en **cajas** para representar el flujo de trabajo.



Paso 2: Tiempo real desde el inicio hasta el final de cada caja

- Después de identificar los pasos, es importante considerar **cuánto tiempo** tarda cada uno.
- Por ejemplo, desde el inicio de la fase de **requisitos** hasta su final, ¿cuántos días se requieren?
- No es obligatorio usar horas para los tiempos en un *Value Stream Map*, pero es más fácil si se hace de forma consistente.
- Se recomienda usar **8 horas por día** y **40 horas por semana**. Esto simplifica la conversión de días a semanas y de semanas a meses.



Paso 2: Tiempo real desde el inicio hasta el final de cada caja

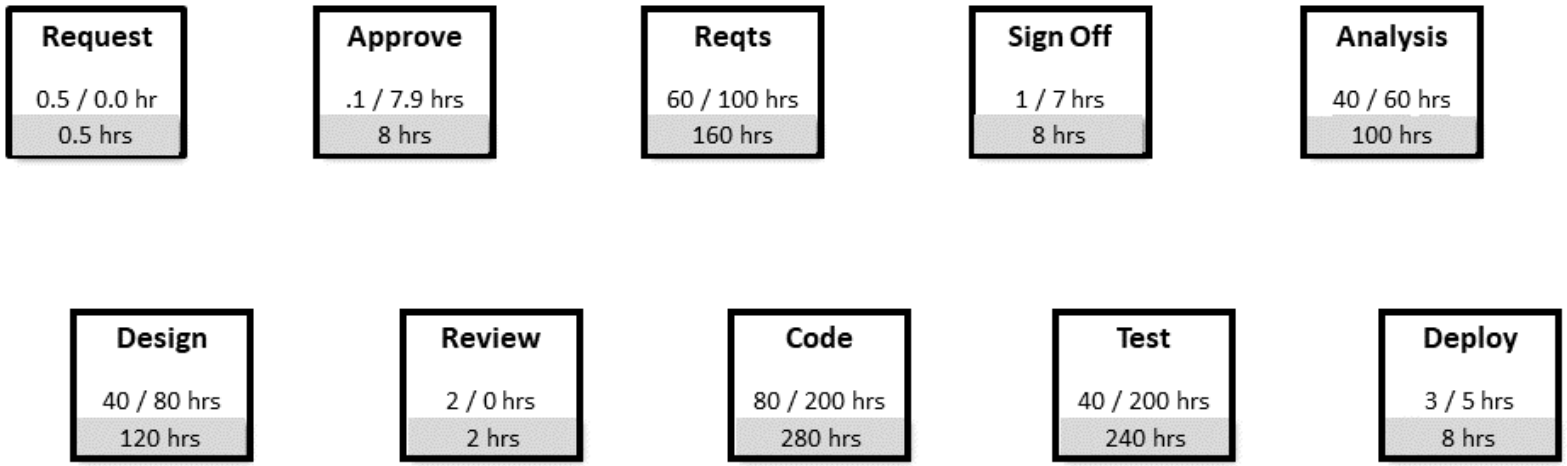


Paso 3: Especificar cuánto tiempo total se dedicó realmente al trabajo

- Ahora llega una parte complicada. Agile sugiere que busquemos **retrasos** en nuestro flujo de trabajo.
- Es difícil ver estos retrasos porque todo el mundo está ocupado trabajando, o existe un estigma o falta de transparencia de comunicar dónde estamos "perdiendo" el tiempo.
- Por ejemplo, si alguien está trabajando en dos proyectos al mismo tiempo, cuando trabaja en uno, **retrasa el otro**.
- Esto es más sencillo de analizar si solo una persona está trabajando. Supongamos que una persona trabaja **60 horas de 160** en la fase de requisitos. Escribiríamos **60 / 100** para indicar 60 horas dedicadas a este proyecto y 100 horas no dedicadas.



Paso 3: Especificar cuánto tiempo total se dedicó realmente al trabajo



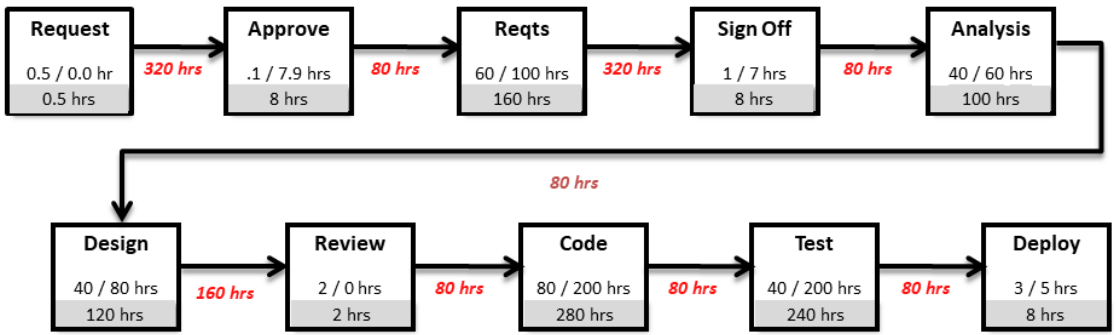
Paso 3: Especificar cuánto tiempo total se dedicó realmente al trabajo

- ¿Y si hubiera **2 personas** trabajando en la fase de requisitos? Digamos que una trabajó **50 horas** y otra **70 horas**. Lo que buscamos es el tiempo trabajado frente al tiempo que no se trabajó.
- La primera persona dedicó 110 horas a otros trabajos, mientras que la segunda dedicó 90 horas a otras tareas.
- La forma correcta de representar esto es **60 / 100**, que indica el promedio del tiempo trabajado frente al no trabajado.
- Ojo: el **esfuerzo** invertido son 120 h (50 + 70), no 60. El promedio mide **dedicación por persona**, no esfuerzo total.

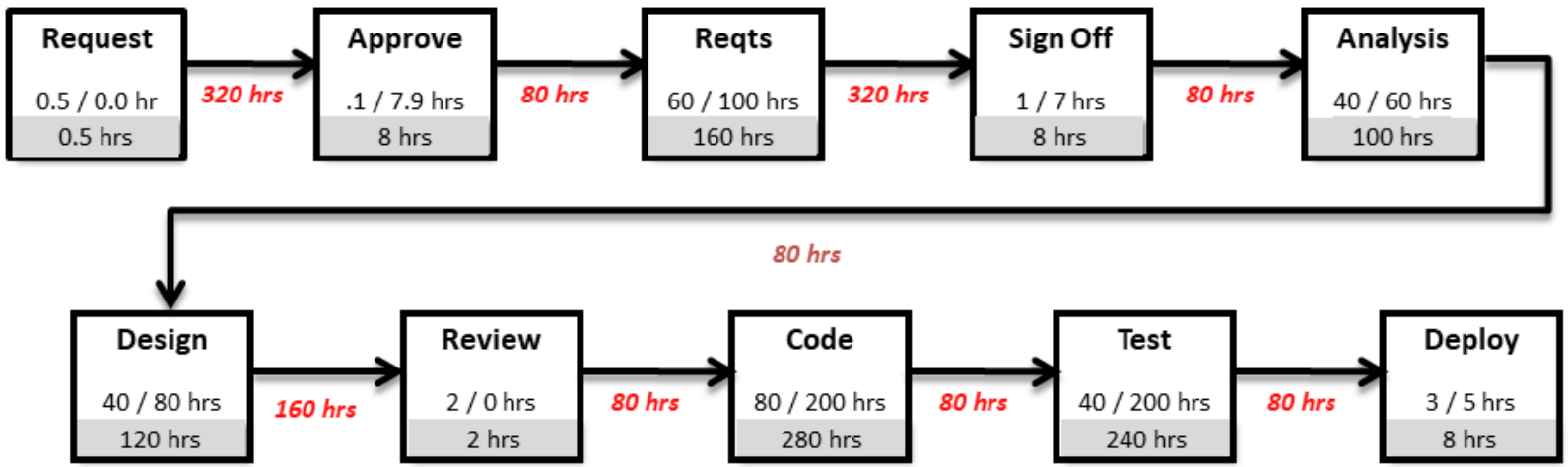


Paso 4: Especificar el tiempo entre las acciones

- Aquí, el proceso se vuelve más sencillo.
- Normalmente, una acción **no comienza inmediatamente** después de que termina la acción anterior.
- Es importante **registrar el tiempo** que transcurre entre el final de una acción y el inicio de la siguiente.
- Esto permite identificar posibles **tiempos muertos** o retrasos entre las etapas del flujo de trabajo.

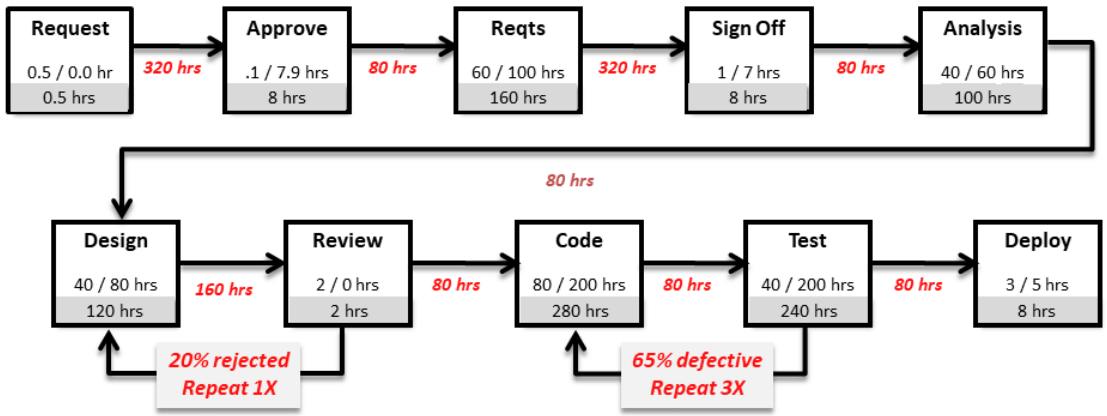


Paso 4: Especificar el tiempo entre las acciones

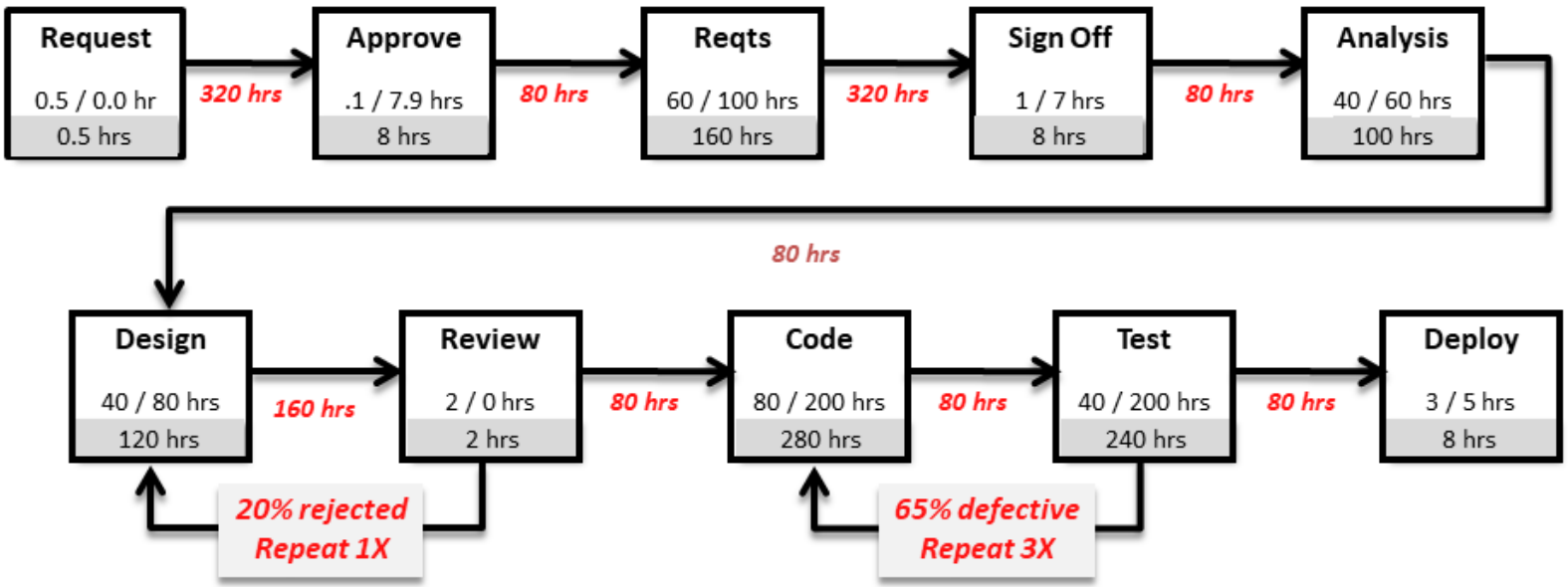


Paso 5: Identificar retrocesos (loop backs) necesarios

- El quinto paso en la creación del *value stream map* es **buscar retrocesos** en el flujo de trabajo.
- Un **retroceso** ocurre cuando una tarea debe volver a una fase previa del proceso antes de poder continuar.
- Es importante identificar **la probabilidad** de que ocurran estos retrocesos.
- Los retrocesos pueden indicar áreas del proceso que requieren **mejora** o donde se producen **retrabajos** innecesarios.



Paso 5: Identificar retrocesos (loop backs) necesarios

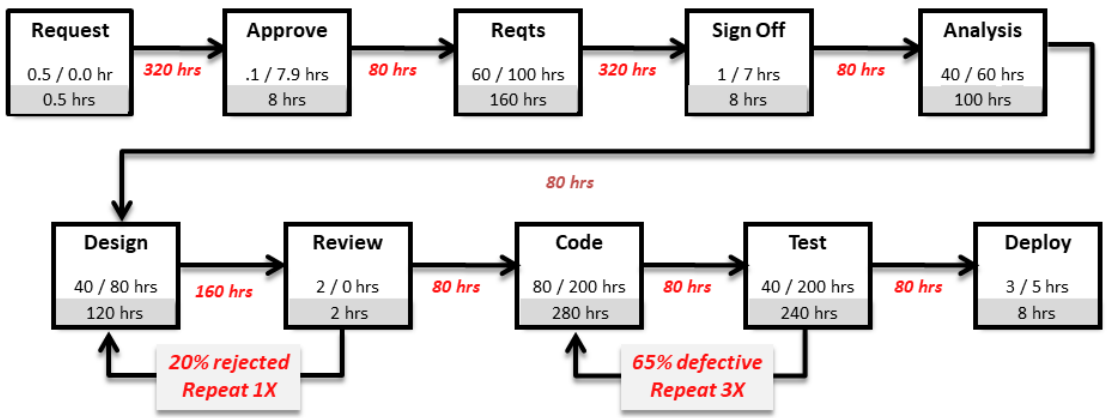


Paso 6: Calcular el tiempo total del proyecto

- El **tiempo total real** (calendar time) de un proyecto es la suma de:
  - Los tiempos en los que se está **trabajando** en cada paso.
  - Los **tiempos entre pasos**, incluyendo cualquier tiempo de **retroceso**.
- El tiempo de calendario sin contar los retrocesos sería:

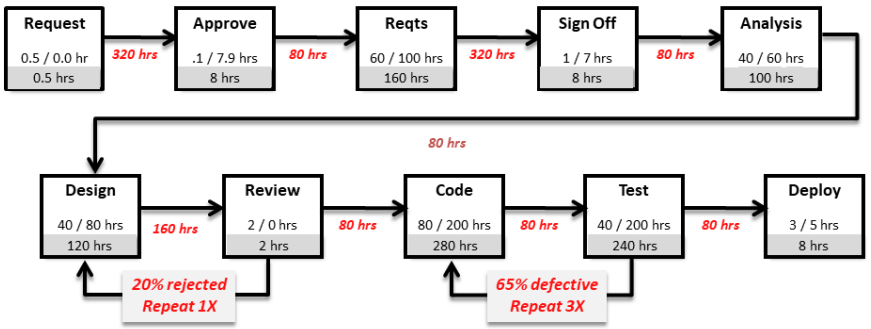
$$0.5 + 320 + 8 + 80 + 160 + 320 + 8 + 80 + 100 + 80 + 120 \quad (1)$$

$$+160 + 2 + 80 + 280 + 80 + 240 + 80 + 8 = 2206.5 \quad (2)$$



Paso 6: Calcular el tiempo total del proyecto

- El tiempo de los retrocesos se calcularía como:
  - $1 \times 20\% \times (120 + 160 + 2) = 56.4$ horas
  - $3 \times 65\% \times (280 + 80 + 240) = 1170$ horas
- La fórmula asume que el **65 %** defectuoso da **3 vueltas**. Con un 65 % de fallo **en cada pasada** serían $0{,}65 + 0{,}65^2 + 0{,}65^3 = 1{,}35$ ciclos, no 1,95.
- Sumando todos estos tiempos, se obtiene un total de **3433 horas**.



Paso 6b: Calcular la eficiencia del ciclo (tiempo trabajado)

- Repetimos el cálculo contando solo el **tiempo trabajado**, retrocesos incluidos, para compararlo con el tiempo total.
- Para el tiempo trabajado obtenemos:

$$0.5 + 0.1 + 60 + 1 + 40 + 40 + 2 + 80 + 40 + 3 = 266.6 \quad (3)$$

- Los tiempos de *loop back* son:

$$1 \times 20\% * (40 + 2) \text{ y } 3 \times 0.65 * (80 + 40) = 8.4 \text{ y } 234 \quad (4)$$

- Sumando todo obtenemos 509 horas.
- La eficiencia del ciclo de proceso es, por lo tanto:

$$\frac{509 \text{ horas}}{3433 \text{ horas}} = 14.8\% \quad (5)$$

- Esto no significa que las personas trabajen solo 1 de cada 7 horas, sino que dedican ese tiempo únicamente a este proyecto.
- Esta métrica es la **Process Cycle Efficiency (PCE)**. En trabajo de conocimiento suele estar entre el **5 %** y el **25 %**.

El mapa de estado actual: qué nos está diciendo

- **3433 h** de calendario frente a **509 h** de trabajo efectivo: unos **429 días laborables** (cerca de 20 meses) para unos **64 días** de trabajo real.
- **1226 h**, el **36 %** del total, son **retrabajo**, casi todo en el bucle entre codificación y pruebas (65 % de defectos).
- **1280 h** son **espera entre cajas**: el ítem no está en ninguna fase.
- El proceso no está dominado por la lentitud del equipo, sino por la **espera** y los **defectos**. De ahí **limitar el WIP** y **calidad garantizada**.
- Esto es el **mapa de estado actual**. Faltaría el **mapa de estado futuro** y el plan de mejora para llegar a él.

Ley de Little en Kanban

- Enunciado: **WIP = Throughput × Cycle Time**. Equivalente: **Cycle Time = WIP / Throughput**.
- En el tablero:
  - **Cycle Time (CT)**: tiempo medio que un ítem pasa en *Doing*.
  - **WIP**: nº de ítems activos en *Doing*.
  - **Throughput**: ítems completados por unidad de tiempo.
- **Condición**: la igualdad solo se cumple si el sistema es **estable** en la ventana de medida, es decir, si entran y salen ítems al mismo ritmo y el WIP no crece ni decrece de forma apreciable. En un tablero que se está atascando, la fórmula da resultados sin sentido.

Ley de Little: un ejemplo con números

- Un equipo de soporte mide su tablero durante **4 semanas**. Cada viernes cuenta las tarjetas que hay en *Doing* y las que ha terminado esa semana.

Semana	1	2	3	4	total
Terminados	11	13	12	12	48
En curso (viernes)	17	19	18	18	media 18

- **Throughput** $= 48/4 = 12$ ítems por semana. **WIP** medio $= 18$ ítems.
- **Cycle Time** $=$ WIP/Throughput $= 18/12 = 1{,}5$ semanas, unos 7 días y medio laborables.

Ley de Little: para qué sirve y qué no dice

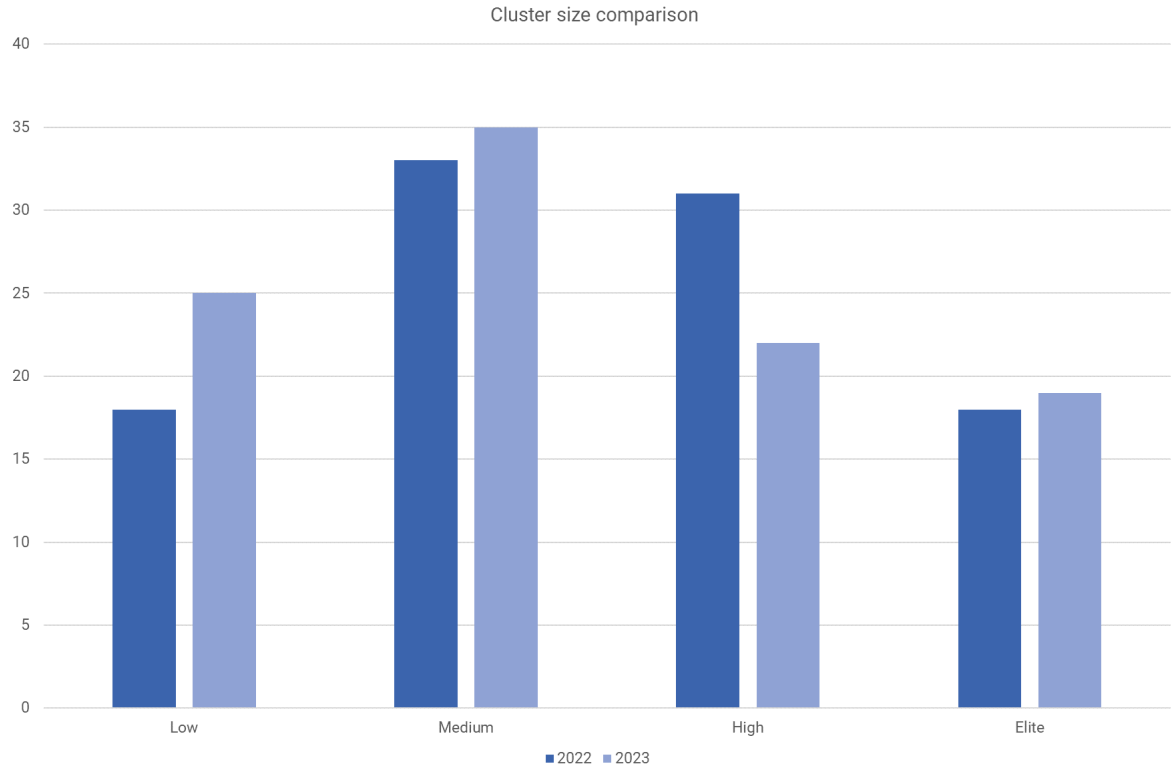
- El WIP se cuenta mirando el tablero y el throughput contando lo que sale cada semana. Con eso ya tenemos el tiempo de ciclo **sin cronometrar ninguna tarjeta**.
- ¿Se puede comprobar que el sistema es estable? Sí: el WIP ronda 18 durante toda la ventana, no crece ni decrece. Si estuviera subiendo semana a semana, el número de 1,5 semanas no significaría nada.
- **La palanca**: si el equipo baja su límite de WIP de 18 a 12 y el throughput se mantiene, el tiempo de ciclo pasa a $12/12 = 1$ semana.
- **Cuidado**: la fórmula **no garantiza** que el throughput se mantenga. Solo relaciona las tres magnitudes, no dice qué causa qué. Que al bajar el WIP el throughput suela subir algo (menos cambio de contexto) es una observación empírica, no una consecuencia de la ley.

Técnicas de ahorro de tiempo en una pipeline software: DevOps

- La adopción de prácticas de **DevOps** puede reducir significativamente los tiempos de desarrollo y mejorar la eficiencia en la entrega de software.
- Algunos ejemplos de objetivos alcanzados con DevOps incluyen:
  - Reducción del **75%** del tiempo medio de entrega desde la integración de código hasta la producción en **3 meses**.
  - Reducción del **33%** del tiempo medio desde la solicitud hasta la entrega a sistemas en producción en **6 meses**.
  - Disminución del **50%** del número de incidentes en producción en **12 meses**.

Ahorro de Tiempo con DevOps

- El **DORA's State of DevOps Report** es un informe anual desarrollado por el equipo de investigación de DORA, que forma parte de Google Cloud. Este informe es uno de los estudios más influyentes en la industria de DevOps, ya que proporciona datos y análisis basados en encuestas a profesionales de TI en todo el mundo.



Principios: Acuerdo

Principios: Acuerdo

- El principio de **acuerdo** en Kanban no solo se refiere a estar de acuerdo sobre la necesidad de un cambio, sino que también es clave para el **arranque** y la **implementación** de cualquier mejora.
- Es necesario un acuerdo efectivo entre quienes:
  - **Solicitan** o **recomiendan** el cambio.
  - **Entienden** el alcance y el **impacto** que el cambio puede tener.
  - **Implementarán** dicho cambio en la práctica.
  - Se verán **afectados** por el cambio en su trabajo diario.

Principios: Acuerdo

- Cuanto más se **superponen** los grupos de trabajo, más **fácil** será llegar a un acuerdo efectivo.
- El acuerdo es un proceso de **búsqueda activa** del acuerdo, más que de esperar a que se alcance de manera natural.
- No es ni un enfoque **top-down** (de arriba hacia abajo) ni **bottom-up** (de abajo hacia arriba), sino encontrar el acuerdo en el **lugar**, **momento** y **circunstancias** adecuadas.
- El **acuerdo es un proceso continuo**, fundamental para asegurar un cambio efectivo y sostenible en la organización.

Principios: Respeto

Principios: Respeto

> *"Showing respect for people does not mean you have to like them, agree with their views, and fail to challenge any half-baked reasoning"*
>
> *Stephen Parry*

- El **respeto a las personas** es un pilar fundamental del *lean development* y uno de los principios básicos de Kanban.
- Reconoce y valora las funciones y esfuerzos de cada miembro del equipo, aunque se puedan tener desacuerdos.
- Un cambio efectivo y duradero se basa en un enfoque respetuoso que **mejora la comprensión** y no busca desacreditar a los demás.

Principios: Respeto

- Pregúntate:
  - ¿Va a aumentar tus posibilidades de éxito si acusas a la gente de que está haciendo un mal trabajo?
  - ¿O si cuestionas su experiencia sin intentar comprender su perspectiva?
- En lugar de imponer cambios, el respeto implica **trabajar juntos para encontrar soluciones** que beneficien a todos.

Principios: Liderazgo

Principios: Liderazgo

> *"Put problems & problem-solvers as close together in time & space as you can; managers provide coaching and scope"*
>
> *Steven J. Spear*

- Kanban fomenta el **liderazgo en todos los niveles** de la organización, no solo en posiciones formales de mando.
- **Cualquiera puede ser líder**: No se necesita un título específico, basta con la iniciativa y la disposición para mejorar.

Principios: Liderazgo

- Se promueve que cada miembro del equipo pueda:
  - **Proponer ideas** y sugerencias de mejora.
  - Participar activamente en la **ejecución** de esas ideas cuando sea posible.
  - Actuar como un **modelo de liderazgo** para sus compañeros, inspirando a otros a contribuir.

Principios: Liderazgo

- El liderazgo en Kanban es una forma de **servir a los demás** y al proyecto, enfocándose en desbloquear obstáculos y facilitar el trabajo.
- Esto no se trata de una jerarquía, sino de un **liderazgo al servicio de un objetivo común**, mejorando continuamente el entorno de trabajo.

Principios: Liderazgo

- **Liderazgo descentralizado**: En Kanban, la toma de decisiones no está limitada a los niveles superiores de la organización.
- Los problemas y soluciones deben ser abordados por quienes **están más cerca del problema**, facilitando la agilidad en la respuesta.
- Esto permite una **rápida adaptación** a cambios y una mayor **resolución de conflictos** dentro del equipo.

Principios: Liderazgo

- Ejemplos de liderazgo efectivo en Kanban:
  - Un desarrollador que identifica un cuello de botella en el flujo de trabajo y propone una forma de optimizarlo.
  - Un tester que, al detectar un problema recurrente, toma la iniciativa para automatizar ciertas pruebas, mejorando la calidad.
- En Kanban, cada miembro del equipo es un **líder potencial** en su área de especialidad.

Core Practices

Core Practices en Kanban

- En Kanban se consideran 6 prácticas principales:
  - Limita el WIP
  - Visualiza
  - Gestiona el flujo
  - Explicita las políticas
  - Introduce la retroalimentación
  - Mejora colaborativamente
- Se suelen resumir en 3: **limita el trabajo, visualiza y mejora**

Limita el WIP

> *"Kanban means stop starting, start finishing"*

- Evitar **exceso** de **multitarea**
- Evitar **sobrecargas** en las siguientes partes de la cadena de proceso (*downstream*)
- El límite del WIP debe establecerse **por consenso** entre todos los implicados en el proyecto
- La tensión creada por establecer un WIP obliga a **discusiones y análisis beneficiosos** para el equipo y el proyecto
- Se puede hacer limitando:
  - El número de trabajos en progreso
  - El número de trabajos por estado de flujo

Limita el WIP

El límite es un concepto fundamental en Kanban.

> *"Don't write more specs than you can code, don't code more than you can test and don't test more than you can deploy."*
>
> *Corey Ladas*

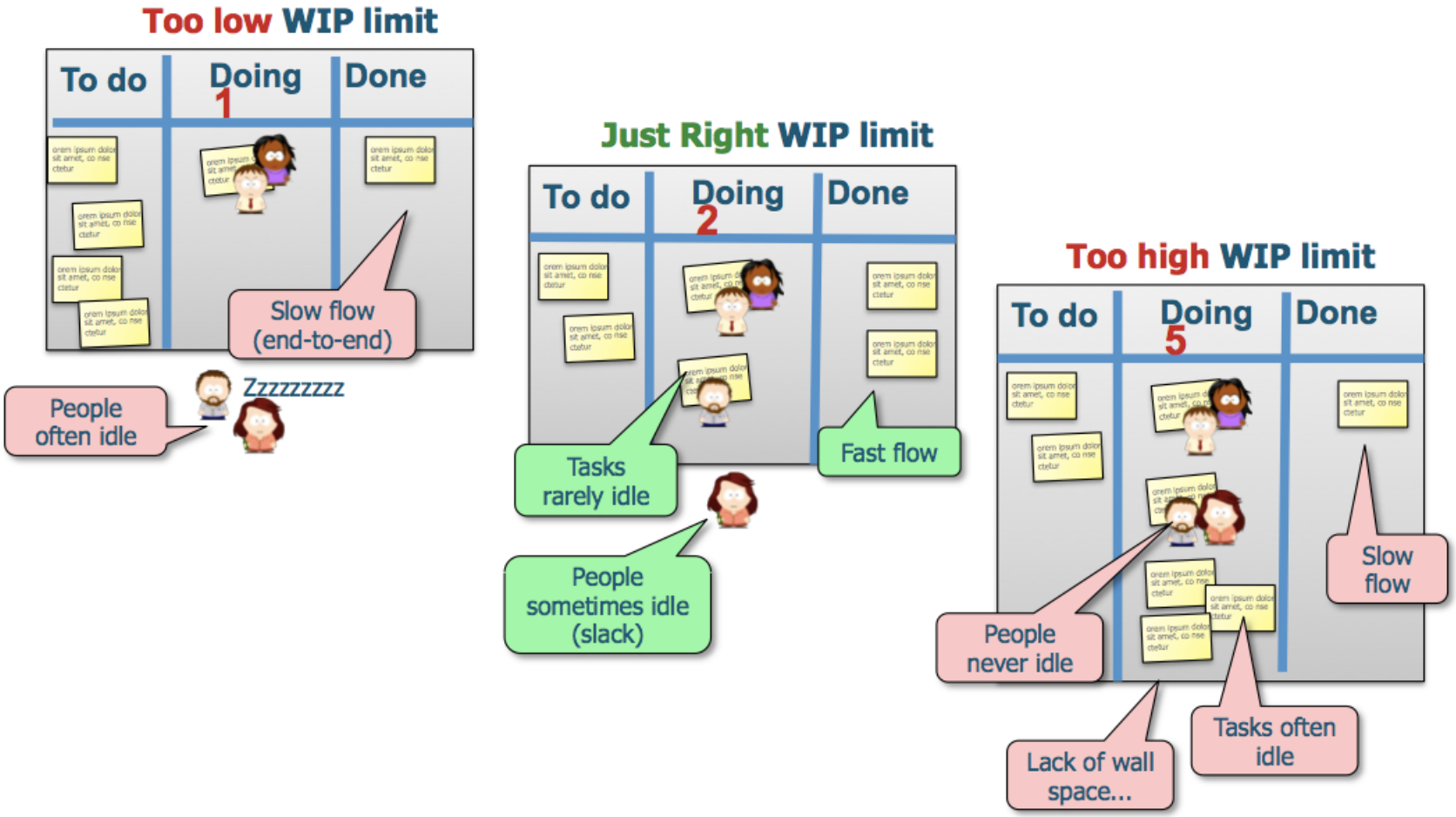
Limita el WIP

- El **límite óptimo de WIP** se consigue a través de **experimentación** y ajustes.
- Una fórmula comúnmente utilizada como punto de partida es:

$$WIP_{lim} = 2 \times \text{número de personas} - 1 \tag{6}$$

- Este cálculo inicial sugiere que el límite debe permitir un flujo de trabajo suficiente sin generar sobrecarga.
- La fórmula busca mantener a los desarrolladores ocupados sin saturarlos, promoviendo un equilibrio entre:
  - **Eficiencia en el flujo de trabajo**
  - **Tiempo de espera mínimo** para tareas
  - Reducción del **cambio de contexto**
- Este valor debe ser evaluado y ajustado periódicamente para adaptarse a la capacidad real del equipo.

Limita el WIP: Trabajos por estado de flujo



Limita el WIP: Trabajos en progreso

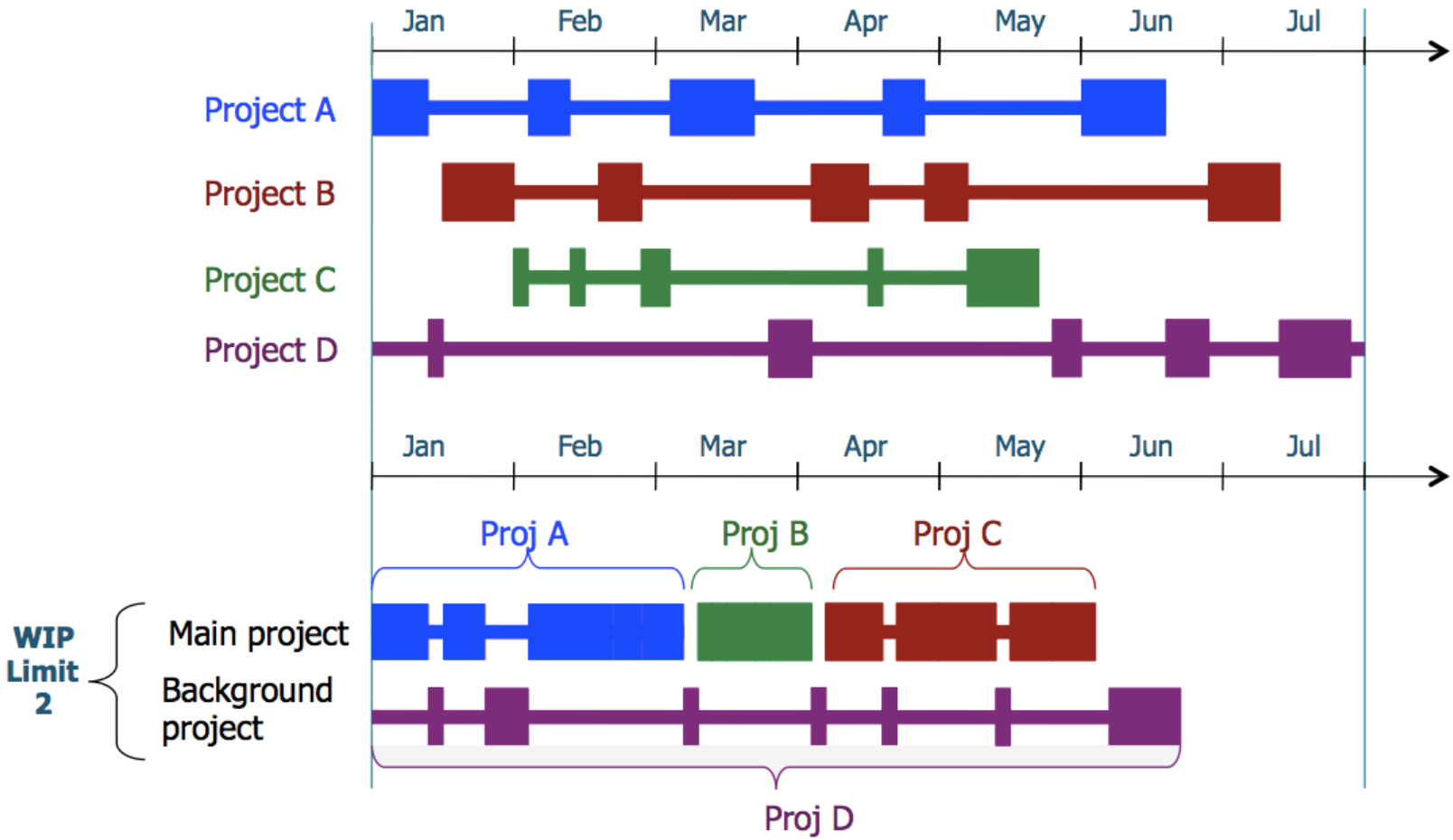
- Podemos considerar un escenario sencillo para entender lo que puede aportar el **WIP**:
  - Un equipo de una sola persona tiene todo lo necesario para desarrollar software, trabajando en tres proyectos simultáneamente.
  - Cada proyecto requiere **30 días** de trabajo en el flujo de "código y pruebas" para completarse.

Limita el WIP

- Si el programador completa los proyectos **uno a la vez**:
  - Los termina en los días **30, 60 y 90**.
  - **Promedio de tiempo de entrega**: **60 días** por proyecto.
- Si el programador trabaja en los tres proyectos a la vez (asumiendo que no hay pérdida por cambio de contexto):
  - Los termina en los días **88, 89 y 90**.
  - **Promedio de tiempo de entrega**: **89 días** por proyecto.
- El enfoque secuencial resulta en un **30% de reducción del tiempo** promedio de entrega.

Limita el WIP

Al limitar el WIP se aumenta el rendimiento:



Limita el WIP

- Sumado a lo previo, en la realidad, cambiar de tarea/proyecto suele ser todavía peor, ya que los proyectos suelen tener **dependencias** y más miembros en el equipo.
- Cambiar constantemente de tarea (**cambio de contexto**) afecta negativamente a la **productividad**.
- Esto puede hacer que la **efectividad** del equipo se reduzca a un **pequeño porcentaje** de su capacidad ideal.
- Proyectos que podrían tomar **un día** pueden tardar **semanas o meses** debido a estos cambios.

Limita el WIP

- El WIP tiende a aumentar cuando se intenta evitar el **"desperdicio"** de tiempo de espera.
- Ejemplo:
  - Un equipo de software suele trabajar de media en **5 a 8 proyectos** al mismo tiempo.
  - Cada proyecto se queda atascado en un punto: falta de respuesta a una pregunta, permisos para bases de datos, etc.
  - Si todos trabajaran juntos, el proyecto podría completarse en **dos semanas**.
  - Sin embargo, el tiempo real de finalización puede extenderse a **dos o tres meses**, gran parte debido a la **espera**.

Limita el WIP

- La espera es uno de los principales **desperdicios** en el desarrollo Lean.
- La práctica correcta es:
  - **Hacer visible** el tiempo de espera.
  - **Identificar** el problema o cuello de botella.
  - **Resolverlo** de inmediato.
- En lugar de esto, muchas empresas **llenan la espera con más trabajo**.
- Esto puede aumentar la **capacidad de trabajo** a corto plazo, pero:
  - **Reduce la capacidad de respuesta**.
  - Complica las **prioridades**.
  - Drena la **energía mental** del equipo debido al cambio constante de tareas.

Limita el WIP

- Limitar el WIP es crucial para:
  - Mantener el **enfoque** del equipo.
  - Reducir el **tiempo total de entrega**.
  - Evitar el **desgaste mental** causado por el cambio de tareas.
- Sin embargo, es importante encontrar un **equilibrio**:
  - Demasiado poco WIP puede llevar a una **baja utilización** de los recursos.
  - Un WIP excesivo alarga los **tiempos de entrega** y afecta la **calidad**.
- El objetivo es mantener un **flujo constante** de trabajo que permita adaptarse rápidamente a los cambios y mantener la **productividad**.

Core Practices: Visualiza

- La visualización del trabajo es clave en Kanban, usando elementos como historias de usuario y tareas desglosadas en notas adhesivas.
- Los ítems de trabajo se representan en un **tablero Kanban**, permitiendo al equipo ver el progreso de las tareas en tiempo real.

Core Practices: Visualiza

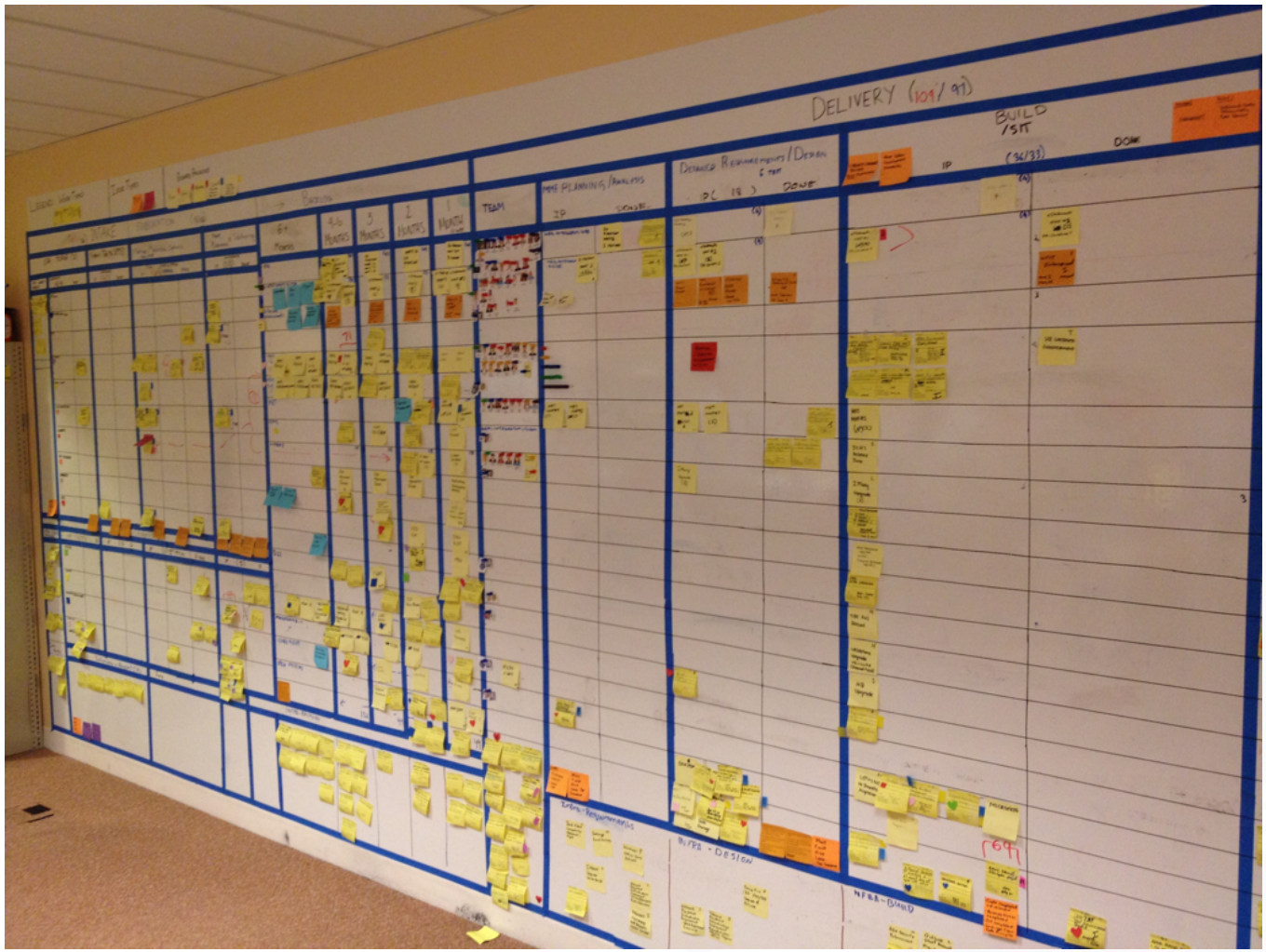
- El tablero Kanban tiene como objetivo principal:
  - Facilitar la **comunicación** y el análisis en las reuniones de pie diarias.
  - Proveer visibilidad del **flujo de trabajo**, ayudando a medir métricas como el número de ítems completados por semana y el WIP.
  - Definir y seguir políticas de flujo y criterios de finalización claros (*Definition of Done*).
- Es una herramienta visual que permite identificar rápidamente cuellos de botella y optimizar el proceso de trabajo.

Visualiza: Tableros Kanban

- Los tableros **evolucionan** con cada equipo
- Los tableros marcan el **flujo** de trabajo de izquierda a derecha
- Deben establecerse las **políticas** de tablero (por ejemplo, cuándo algo está listo para test, cuándo está acabado, etc.)
- Pueden tenerse cajas distintas según el **tipo de trabajo** (features, stories, bugs)
- Hay quien trabaja con **buffers**: tareas a la espera de entrar en una fase
- Deben gestionarse las **estadísticas** de avance

Visualiza: Tableros Kanban

Intentamos evitar cosas como esta, es importante una visualización fácil para los desarrolladores y stakeholders!!



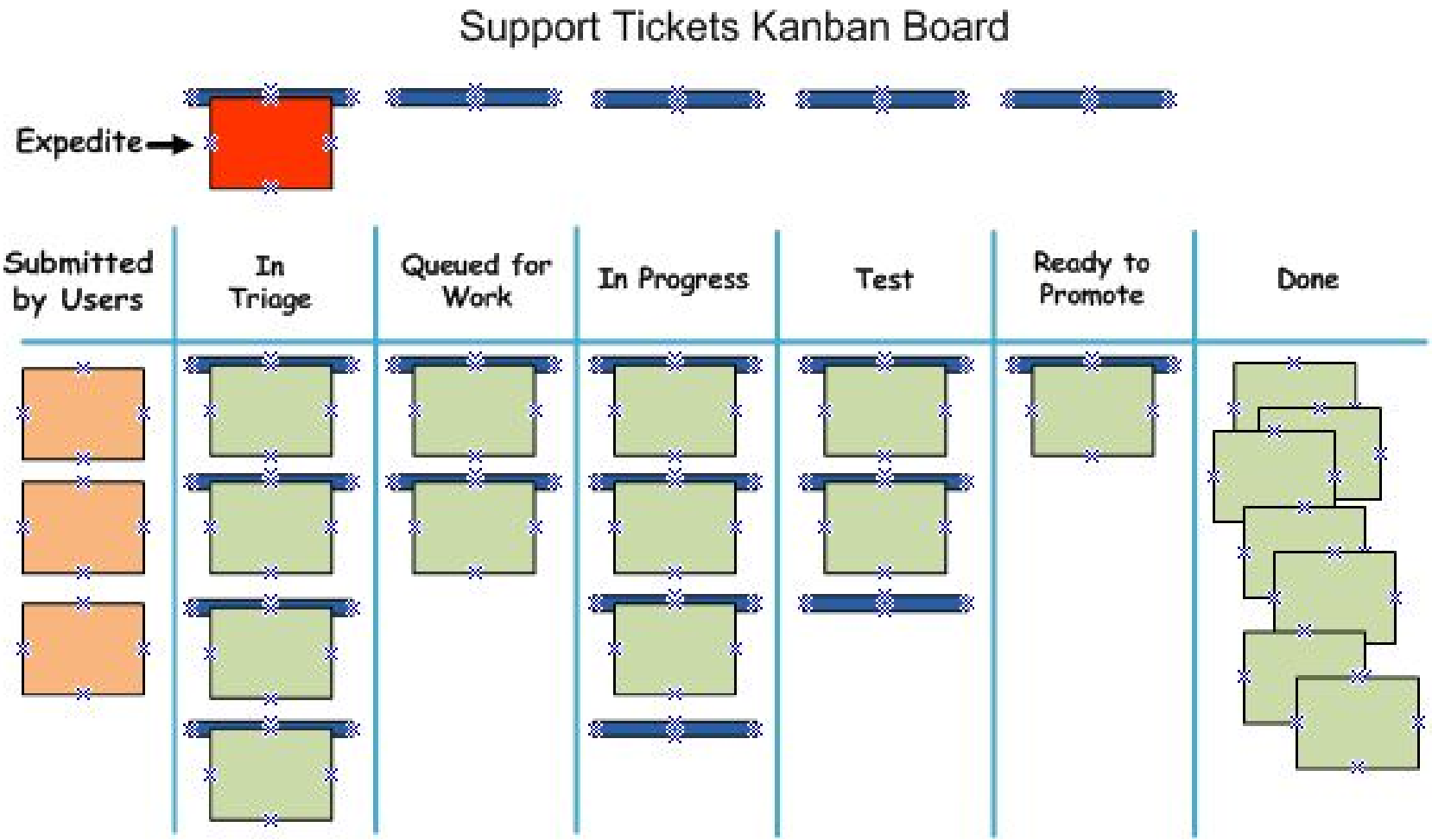
Prioridades en tableros Kanban

- Las **clases de servicio** ayudan a categorizar y priorizar diferentes tipos de trabajo según su urgencia e impacto en el proyecto.
- Existen varias clases comunes de servicio:
  - **Urgente (Expedite)**: Trabajos críticos que se priorizan por encima de todas las demás tareas.
  - **Estándar**: Tareas que siguen el flujo de trabajo regular.
  - **Fecha Objetivo**: Tareas con fechas de entrega específicas.

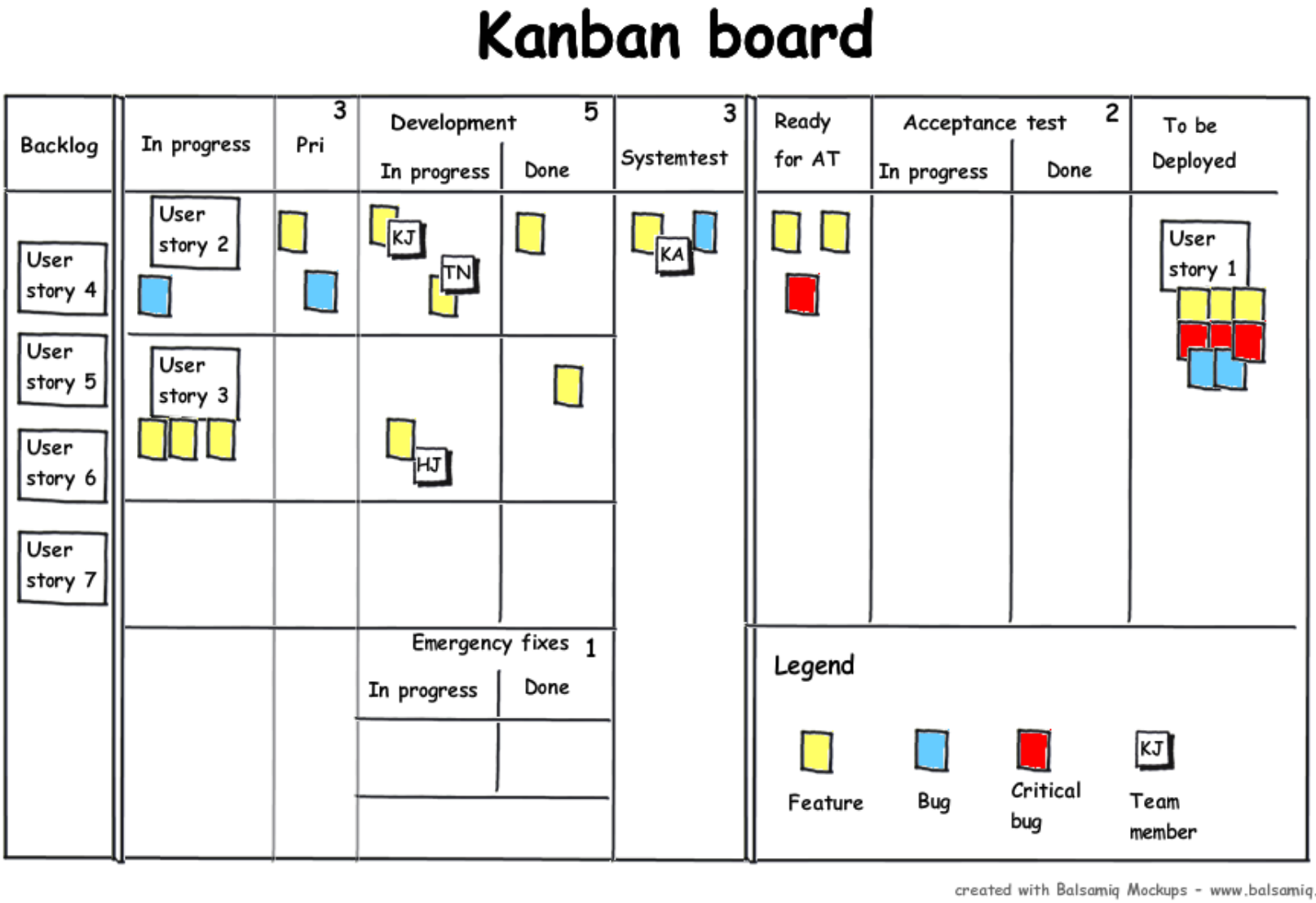
Prioridades en tableros Kanban

- Las tareas de clase **expedite** tienen prioridad máxima y deben ser atendidas de forma inmediata.
- Estas tareas se colocan en un carril especial de alta prioridad en el tablero Kanban, conocido como **fast lane**.
- El objetivo es minimizar el **tiempo de ciclo** para que las tareas urgentes se completen rápidamente y sin bloqueos.
- Ejemplo: Corregir un fallo crítico en producción que afecta a la experiencia del usuario.

Prioridades en tableros Kanban



Visualiza: Ejemplo de Tablero Kanban



Ejemplo de Tablero Kanban

- El tablero Kanban visualiza el flujo de trabajo y las tareas actuales. En el ejemplo de la diapositiva anterior:
- **Columnas**: representan las distintas etapas del proceso de desarrollo (e.g., Backlog, Desarrollo, Pruebas de Sistema, Listo para Deploy).
- **Tarjetas Kanban**: cada tarjeta representa una tarea específica como una *user story*, un bug, o una nueva característica.
- **Columna "Pri"**: contiene las 3 tareas más prioritarias para que el equipo pueda enfocarse y seleccionar el próximo trabajo.

Ejemplo de Tablero Kanban

- **Estación de Desarrollo**: tiene un límite de 5 tarjetas Kanban para evitar la sobrecarga del equipo.
- **Pruebas de Sistema (System Test)**:
  - Puede contener hasta 3 tarjetas Kanban.
  - Asegura que cada tarea sea probada en un entorno completo antes de avanzar.
  - Previene la acumulación de cambios no probados en el sistema.
- Utilidad de los límites: Cuando ocurre un cuello de botella en pruebas, el desarrollo se detiene, obligando al equipo a enfocarse en resolver el bloqueo.

Visualiza: Beneficios de Limitar el WIP

- Limitar el WIP permite al equipo mantener un enfoque claro y evitar la multitarea excesiva.
- Promueve la colaboración entre los miembros del equipo para resolver cuellos de botella y avanzar en el flujo.
- El sistema de **pull** ayuda a prevenir la sobrecarga de nuevas tareas antes de completar las existentes.
- **Resultado**: menor tiempo de entrega y mayor calidad en el producto final.
- Además, si queremos incluir columnas para testing, debemos comprometernos a:
  - Todas las tareas deben ser **testeables** y orientadas al valor del cliente.

Visualiza: Manejo de los bugs

- No importa cuán sólido sea el proceso o cuán talentoso sea el equipo: **siempre aparecerán bugs**.
- Tratar los bugs de manera eficiente es **esencial** para construir un producto de alta calidad.
- Los bugs deben ser tratados como tareas de **primera clase**, al igual que las características, para evitar su acumulación.
- Si los bugs no se manejan rápidamente, el software se **degradará** a un ritmo mayor de lo necesario.

Visualiza: Manejo de los bugs

- Es preferible resolver los bugs lo antes posible, ya que esto reduce el impacto a largo plazo en la calidad del producto.
- **Tratamiento en Kanban**:
  - Los bugs se deben gestionar de forma visible en el tablero Kanban.
  - Asignar prioridad y tratarlos con la misma seriedad que otros ítems del flujo de trabajo.

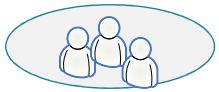
Visualiza: Tiempo de Entrega

- Un concepto importante en Kanban es el **tiempo de entrega** o *lead time*.
- Se refiere al tiempo que tarda una funcionalidad en pasar por todo el proceso, desde su inicio hasta su finalización.
- Aunque Kanban no tenga iteraciones fijas o estimaciones estrictas, como Scrum, el lead time permite hacer **predicciones**.
- Ejemplo de predicción: "Podemos entregar la historia de usuario X en **7 días**".
- Una rama del desarrollo SW defiende que el lead time se vuelve una métrica más **confiable** para predecir la entrega de trabajo que los plazos generales de iteraciones.

Visualiza: Tableros Kanban para equipos Scrum

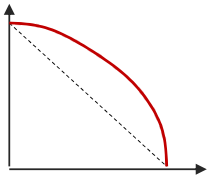
Scrum team
applying WIP limits

Context

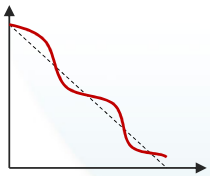


Scrum team

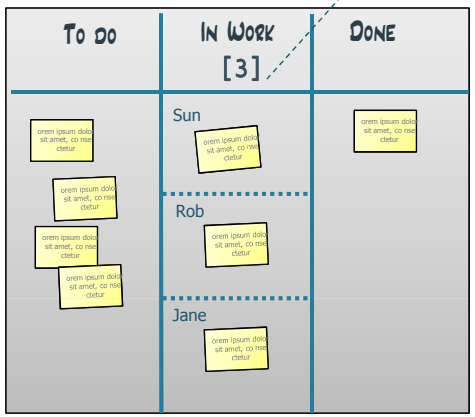
Why? To trigger a shift from
a burndown like this..



To something more like:



Work in progress limit

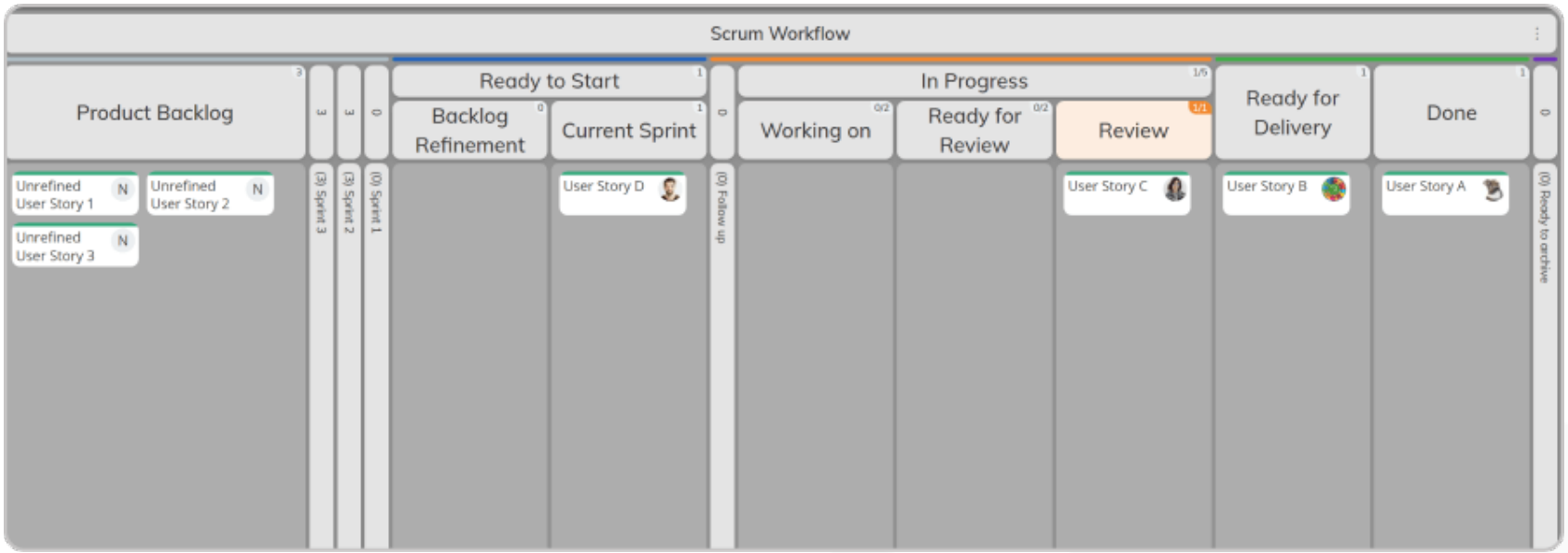


crisp

Mattias Skarin, 2010

Visualiza: Tableros Kanban para equipo Scrum

Vamos a ver un ejemplo de cómo construir un tablero **Kanban** pensado para la metodología **Scrum**

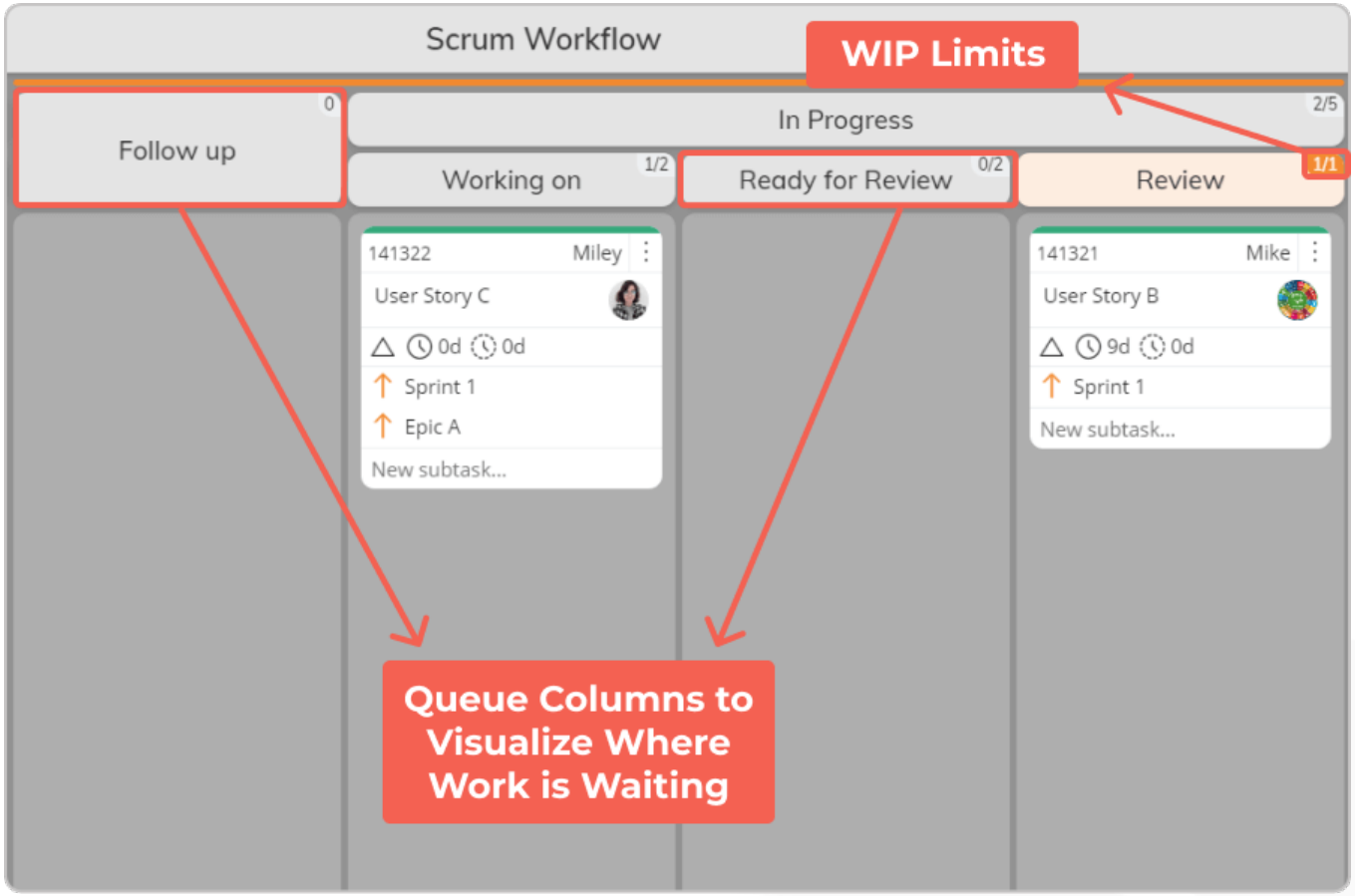


* Incluye contenido modificado bajo artículos de Kanbanize

Visualiza: Tableros Kanban para equipo Scrum

- **"In Progress"**:
  - Aquí es donde se realiza el trabajo después de acordar un **objetivo de sprint** y comprometerse con un conjunto de historias de usuario.
- Ejemplo de columnas del proceso:
  - **"Working on"**: Representa las tareas activas en desarrollo.
  - **"Ready for Review"** y **"Review"**: Pasos para la revisión de las tareas completadas.
- Buena práctica: Etiquetar las columnas donde el trabajo está inactivo como **"colas"** para medir la eficiencia del proceso.
- **Límites de WIP**: Aplicar límites a las columnas de actividad y de cola para evitar cuellos de botella durante las iteraciones.

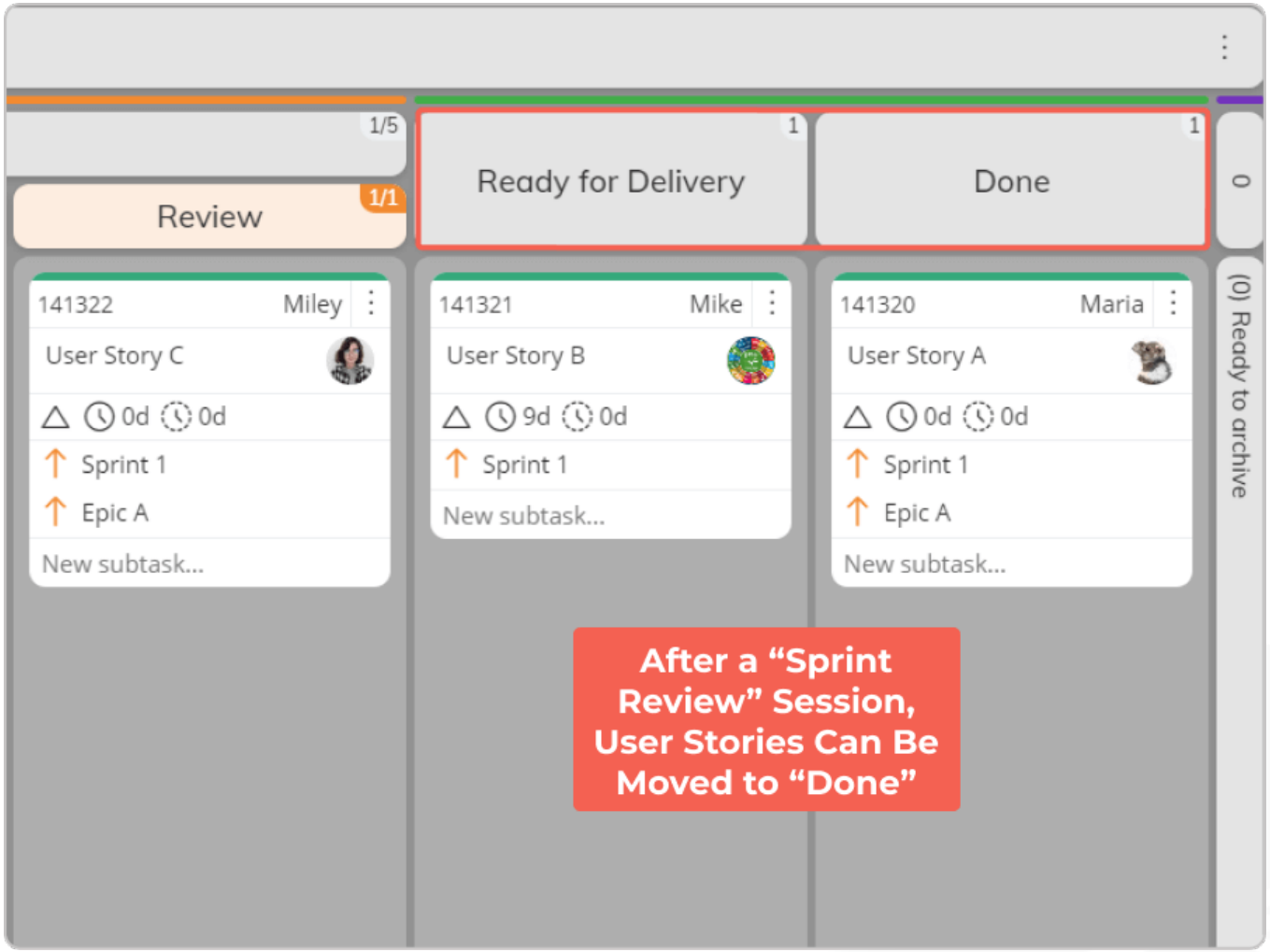
Visualiza: Tableros Kanban para equipo Scrum



Visualiza: Tableros Kanban para equipo Scrum

- **Área "Done"**:
  - Representa la fase final del tablero Kanban donde el equipo entrega el trabajo.
- Como vimos, en un **Scrum** teórico, al finalizar cada sprint se realiza una **"Sprint Review"**.
- Ejemplo: Desglosar el área "Done" en dos columnas:
  - **"Ready for Delivery"**: Punto de compromiso donde se acumulan los elementos de trabajo completados.
  - **"Done"**: Después de la evaluación en la "Sprint Review", las historias de usuario pueden declararse como **"done"** y moverse a esta columna.
- Esta separación ayudaría a gestionar el proceso de entrega continua y facilita la revisión de los elementos terminados antes de su despliegue final.

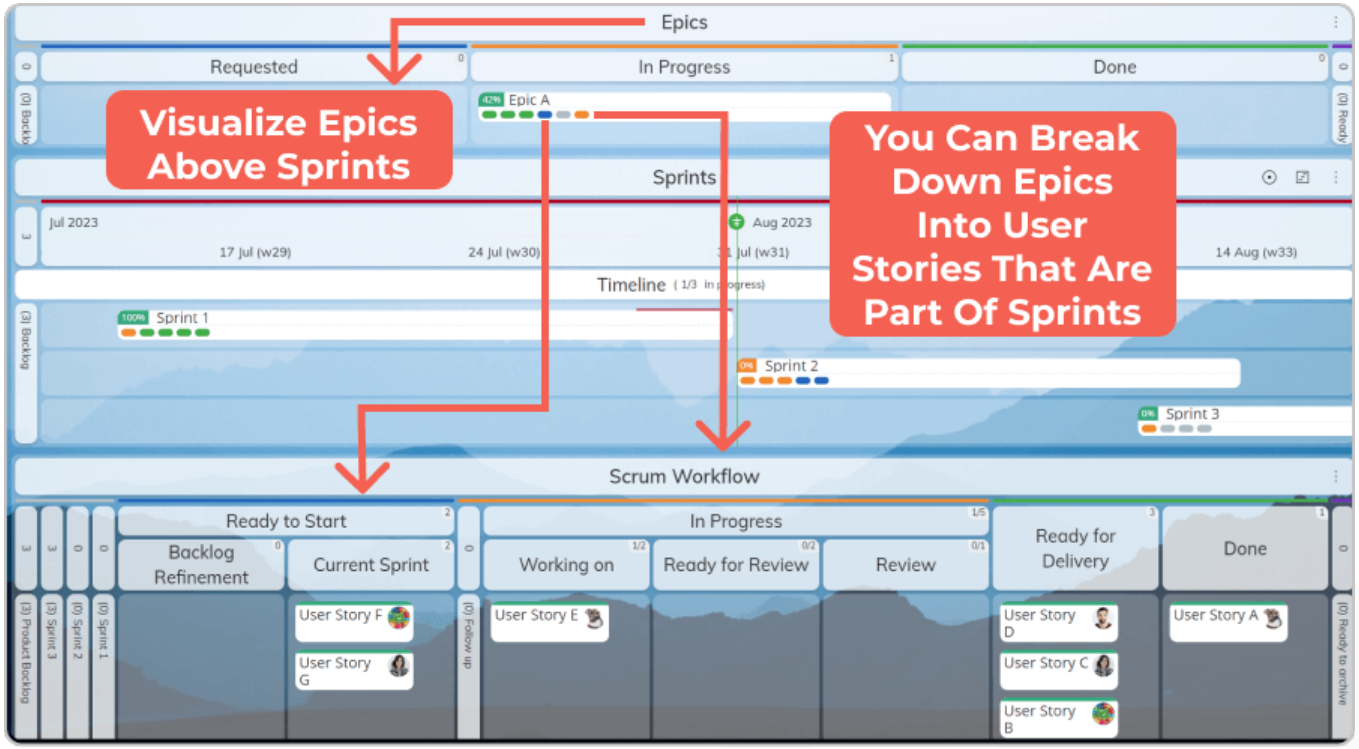
Visualiza: Tableros Kanban para equipo Scrum



Visualiza: Tableros Kanban para equipos Scrum

- Para gestionar productos completos, normalmente existen distintas **releases**.
- En el contexto de Scrum, cada release se compone de varios **sprints**, desglosados en **historias de usuario** y/o **epics**.
- En un tablero Kanban, esta estructura se puede reflejar apilando múltiples flujos de trabajo o conectando tableros separados para una estructura a gran escala.

Visualiza: Tableros Kanban para equipo Scrum



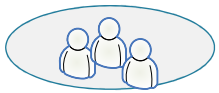
Visualiza: Tableros Kanban para equipos Scrum

- **Sprints**: Encima del flujo de trabajo Scrum, se puede agregar un flujo de trabajo de línea de tiempo para visualizar la duración de los sprints (p. ej., 2 semanas).
- Esto facilita la conexión de **historias de usuario** a un sprint y permite desglosar los sprints en historias.
- La estructura ayuda a **rastrear automáticamente** el progreso de los sprints a medida que se completan las historias.
- Otro enfoque posible es visualizarlas encima de los Sprints.
- Dependiendo del escenario, las épicas se pueden desglosar en historias y distribuirlas a lo largo de varios sprints.

Tablero Kanban basado en TDD

Development team using defined process

Context

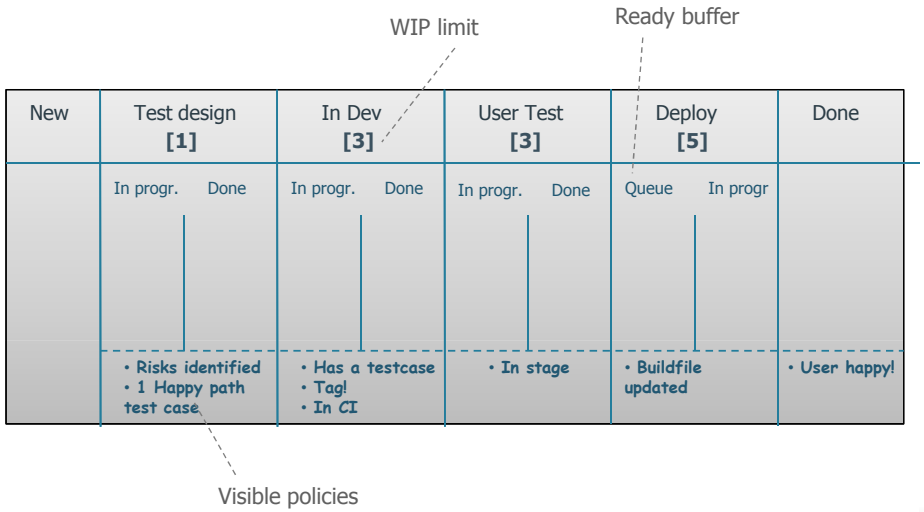


Development team combined with specialists

Stakeholders:
- Product owner / Project manager

Pro:
- Polices are clear
- WIP in each step
- Ready buffer's from which next step can pull work

Cons:
- New column can get messy if no person maintains it



crisp

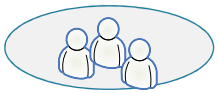
A buffer is a trade-off between cycle time and variance absorbtion

Mattias Skarin, 2010

Tableros Kanban de un mismo equipo y distintos clientes

Development team with multiple clients

Context

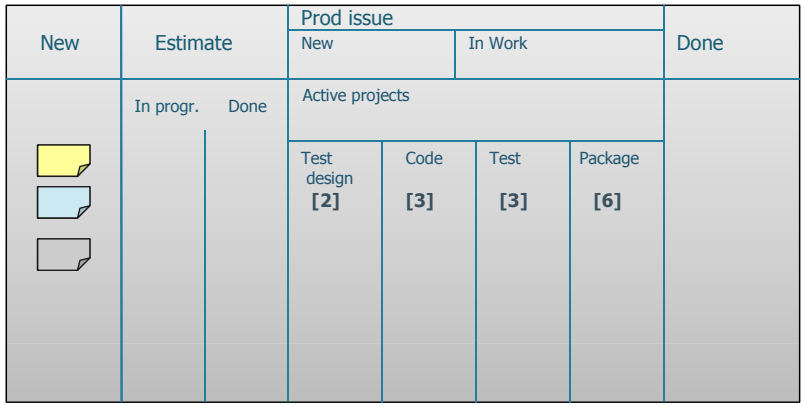


Custom solutions dev team with project manager

Stakeholders:
- Customer A
- Customer B
- Other teams
- Platform architects

Pro:
- Projects and features visible
- WIP in each step
- Estimations can be regular /or "on need" triggered event

Cons:
- Tempting to default features to "time constrained" (even though there really isn't any costly delay consequence)



Classes of services in use:
- Time constrained feature
- Ordinary feature
- Bug
- Fixing tech debt

To risk balance your portfolio, limit the amount of each category allowed on the board at any time

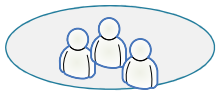
crisp

Mattias Skarin, 2010

Basado en predicción de completado basado en semana

Development team with completion prediction

Context

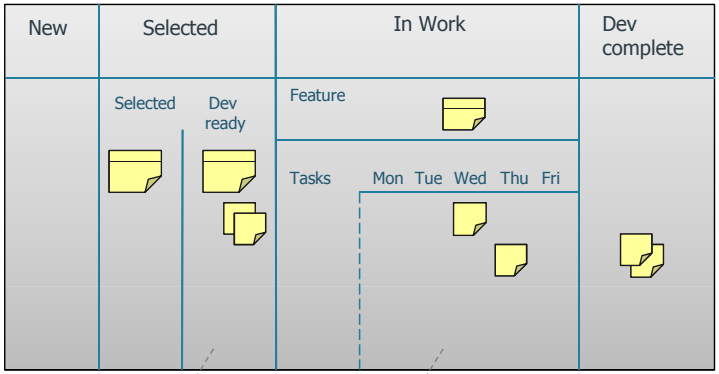


Development team

Stakeholders:
- Product owner / Project manager

Pro:
- Completion date visible
- Learning of prediction towards cycle time

Cons:



If estimated size > 5d task is broken down further

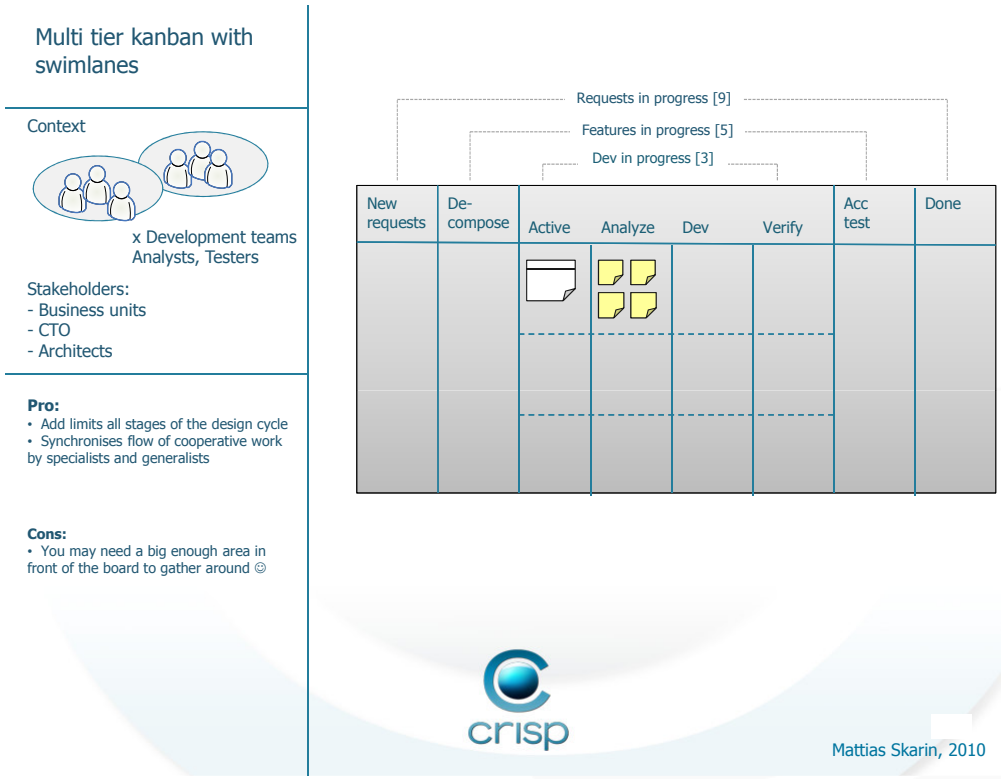
When developer starts a task it is placed on the day they think it will finish. Each day, this prediction is updated.

crisp

Mattias Skarin, 2010

Tableros Kanban con swimlanes

Las **swimlanes** son divisiones horizontales en un tablero Kanban que ayudan a organizar y categorizar el trabajo en función de criterios específicos. Facilitan la visualización de diferentes tipos de trabajo o proyectos paralelos.



Tableros Kanban para sysadmins

System administration

Context

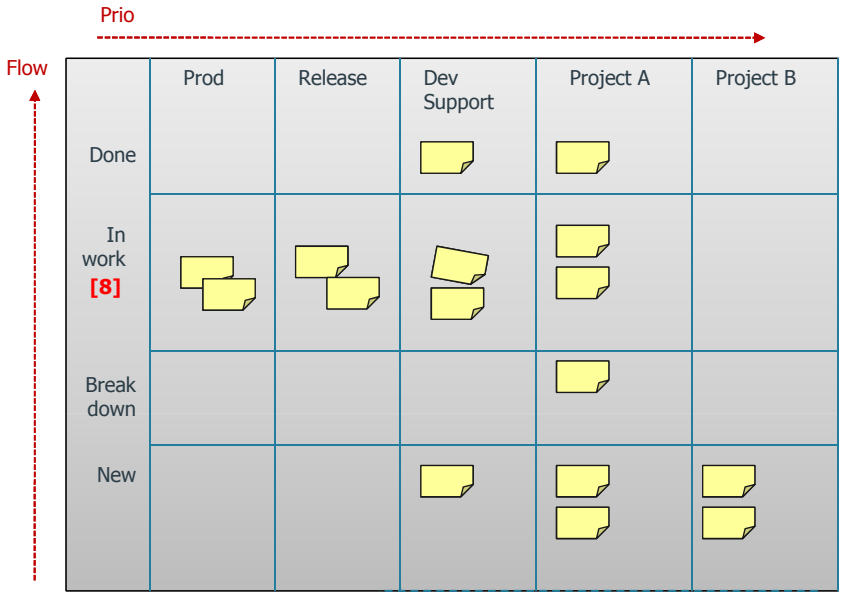


System administrator team supporting development and production

Stakeholders:
- Production site
- Development teams
- Internal projects
- Testers

Pro:
- Course grained prio visible
- WIP balanced across work types
- Visible learnings opportunities for team members in maintainance and project work

Cons:
- Newly arrived reguests can get messy if no person maintains it



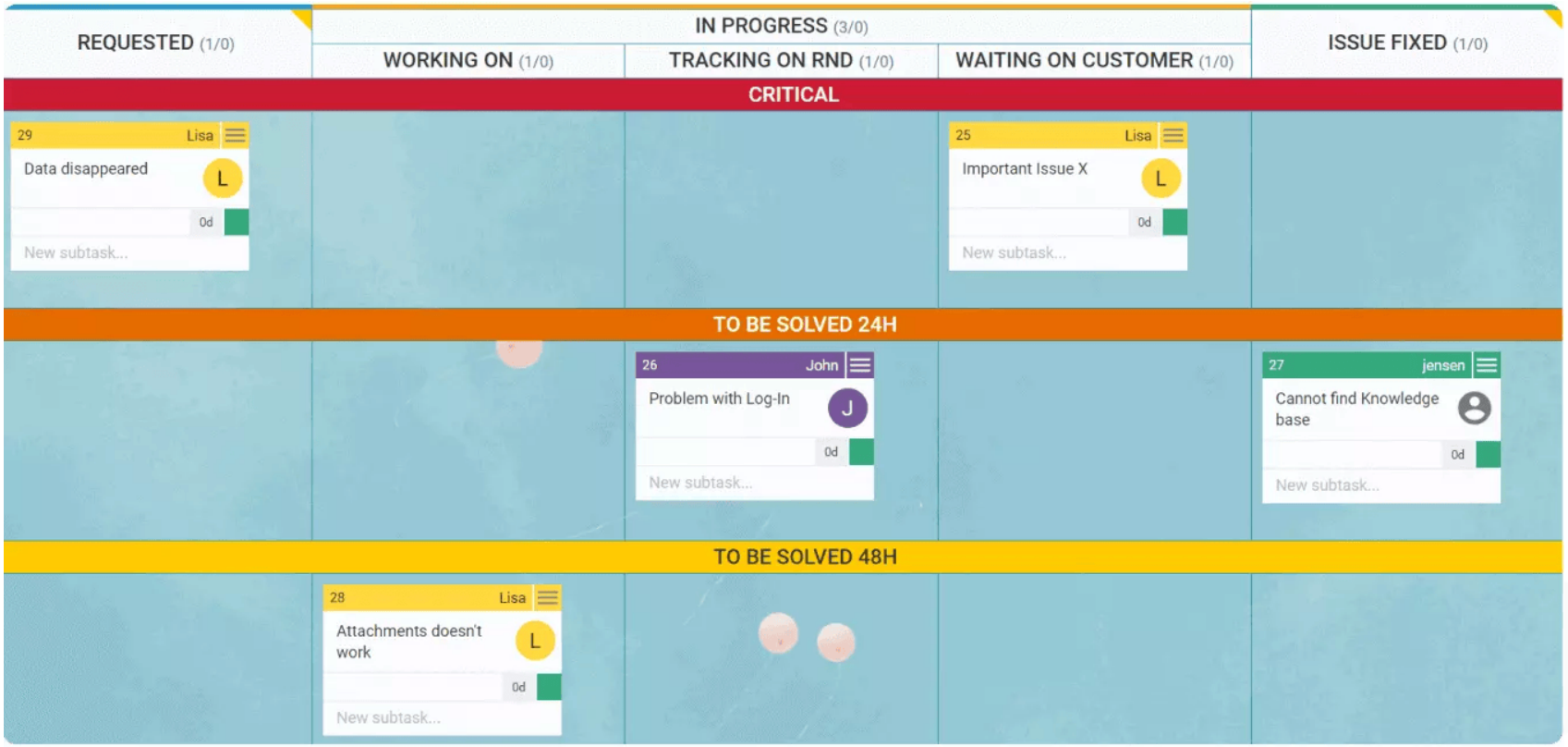
crisp

Mattias Skarin, 2010

Tableros Kanban para support systems

- Los **equipos de soporte** suelen tener flujos de trabajo muy dinámicos debido a la llegada y resolución rápida de tareas, como los tickets de soporte.
- Es crucial contar con una solución centralizada para rastrear todas las solicitudes de clientes externos o solicitudes internas.
- En estos tableros, es habitual el uso de **swimlanes** para dividir el trabajo según la prioridad o el tiempo necesario para resolver los problemas.
- Normalmente, se suelen estructurar en dos tableros: **first-level and second-level support**.

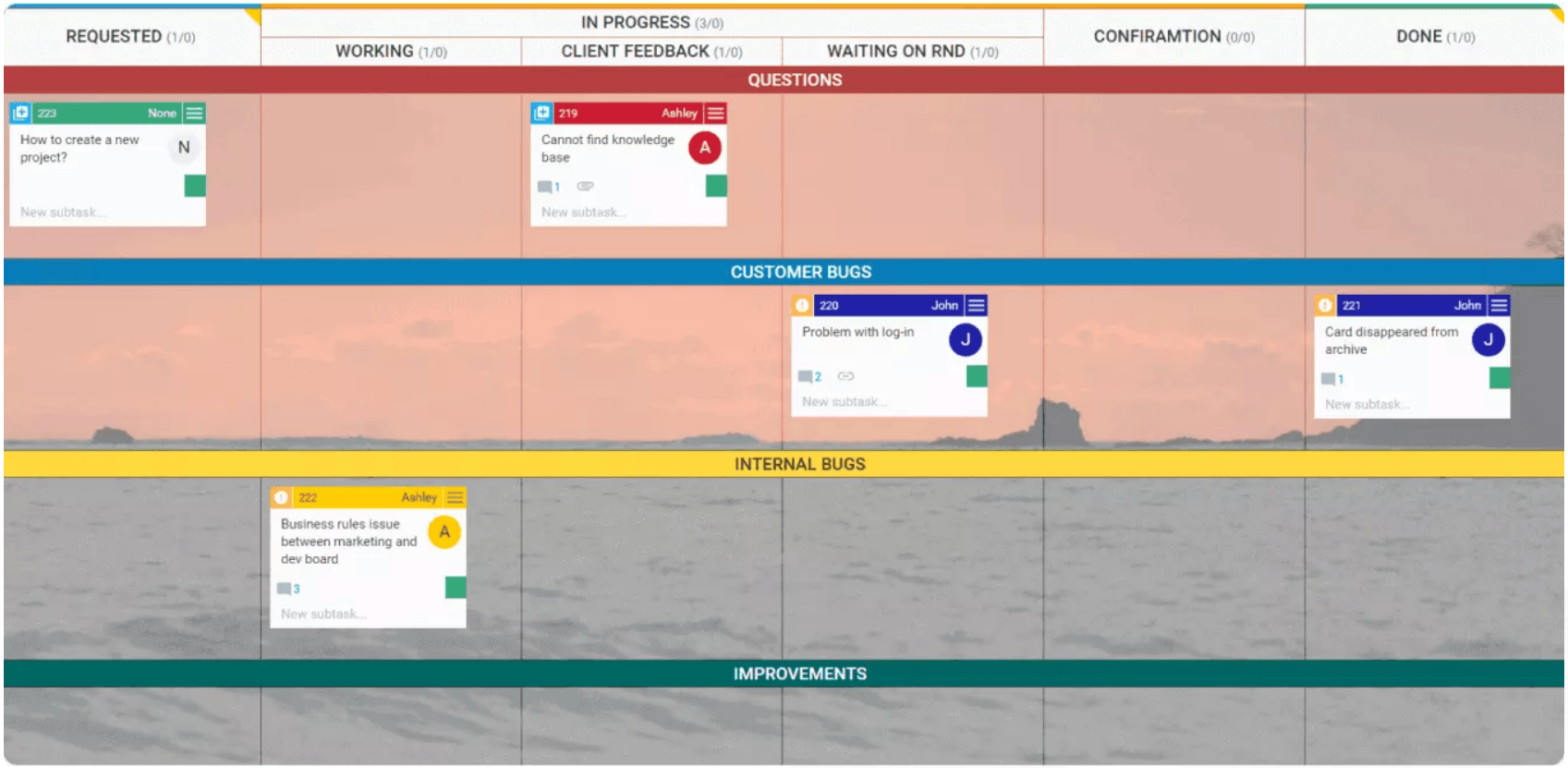
Tableros Kanban para equipo Scrum



Tableros Kanban para support systems: First level

- El **first-level support** se encarga de resolver problemas básicos de los clientes.
- Los miembros del equipo recopilan la mayor cantidad de información posible del cliente para resolver el problema.
- Los clientes pueden ser usuarios internos, por lo que se recomienda usar **swimlanes** para dividir los tickets de soporte externo e interno.
- Se pueden implementar etapas de proceso típicas, como una columna de **Client Feedback**.

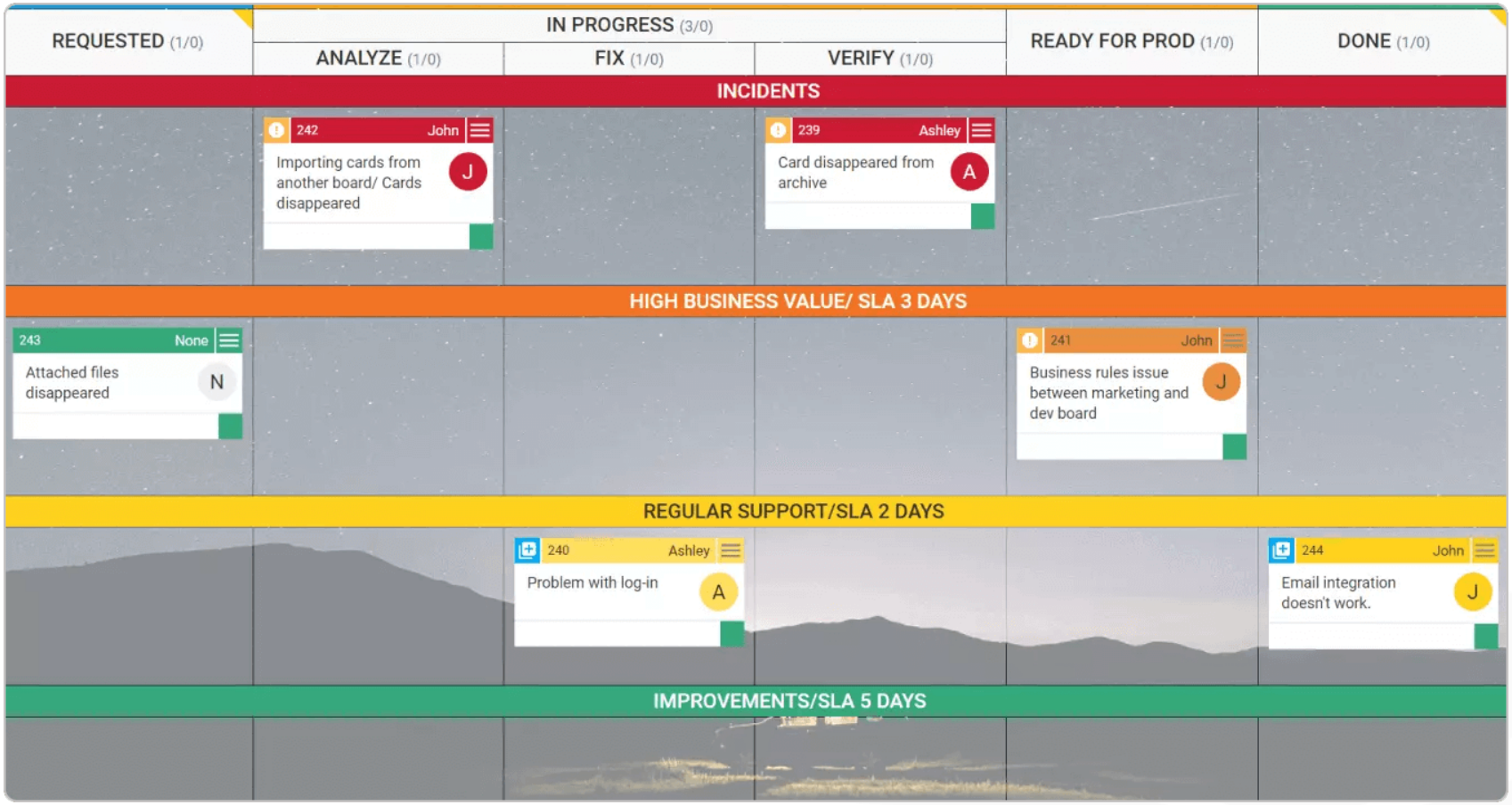
Tableros Kanban para support systems: First level



Tableros Kanban para support systems: Second level

- El **second-level support** incluye tareas más complejas y de mayor importancia, realizadas por miembros con conocimiento avanzado del producto o servicio.
- Se pueden utilizar **swimlanes** para diferenciar tareas de alto valor para el negocio de tickets de soporte regular.
- Es común usar la categoría de **Incidentes/Expedites** en la parte superior del tablero para destacar tareas urgentes.

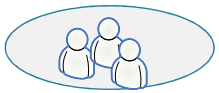
Tableros Kanban para support systems: Second level



Tableros Kanban para equipo de Marketing

Marketing team

Context



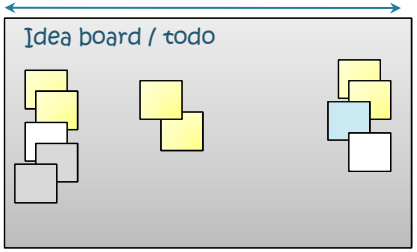
Small marketing team
PR, web, graphics, blog

Stakeholders:
- CEO
- Sales

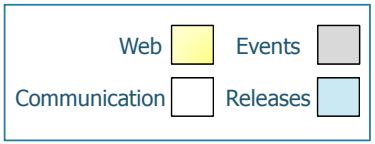
Pro:
- Ideas repriotitization and aging visible
- Visual progress of combined work

Cons:
- Over administration?

Ice (popcicles) ⟷ Hot (dogs)



Classes of services



Marketing kanban

In progress	@ Third party	Under Validation	(well) Done

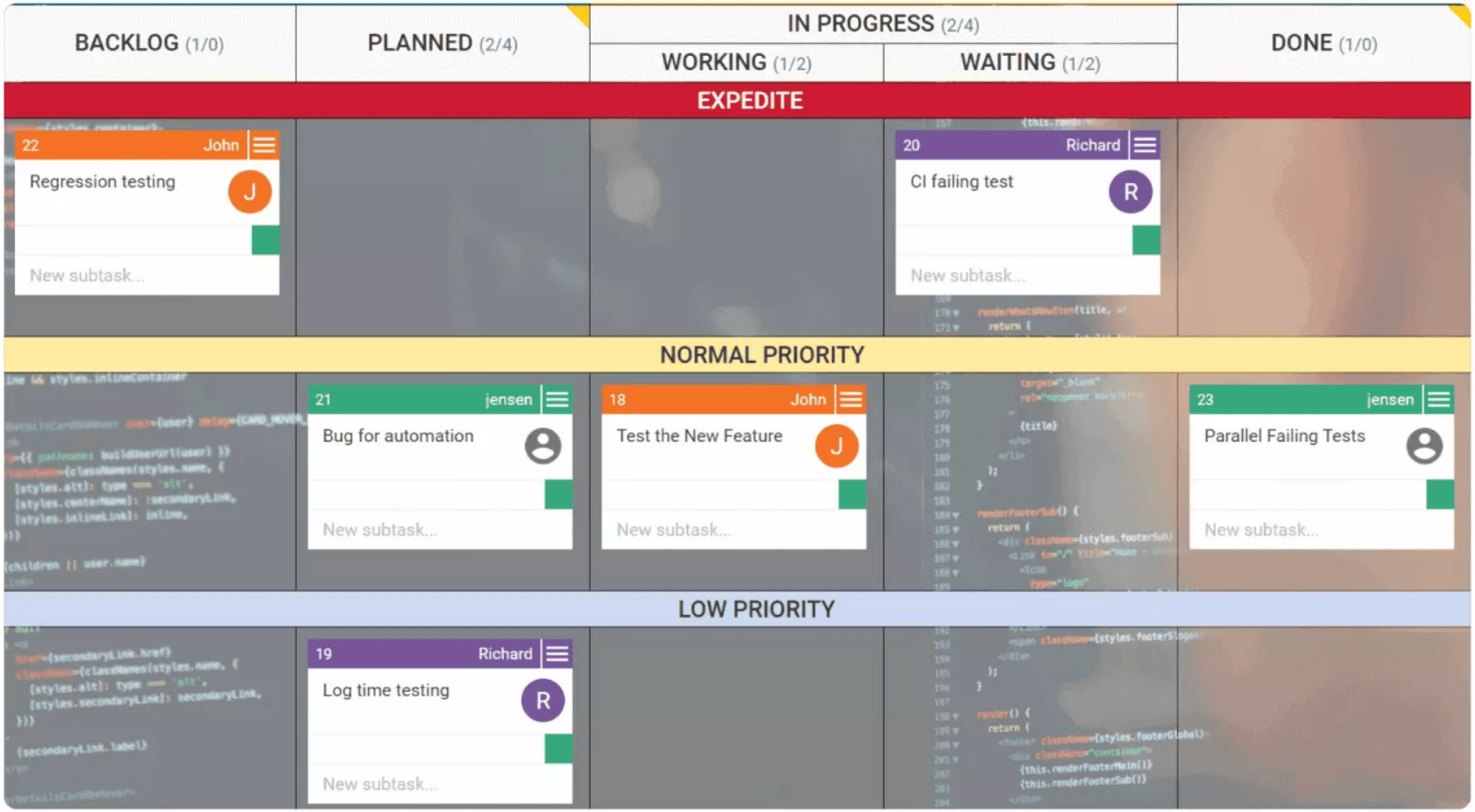
crisp

Mattias Skarin, 2010

Tableros Kanban para equipos de QA

- Los equipos de **QA** *(aseguramiento de calidad)* están estrechamente relacionados con los equipos de desarrollo.
- Ambas partes son cruciales para el éxito del proceso de desarrollo de productos y suelen trabajar de manera interdependiente.
- Sin embargo, es común que los equipos de desarrollo y QA trabajen en el mismo tablero Kanban, o, si son diferentes, que puedan ver sus respectivos tableros.
- Las **swimlanes** también son útiles aquí como herramienta de priorización, permitiendo localizar y seguir fácilmente problemas críticos.

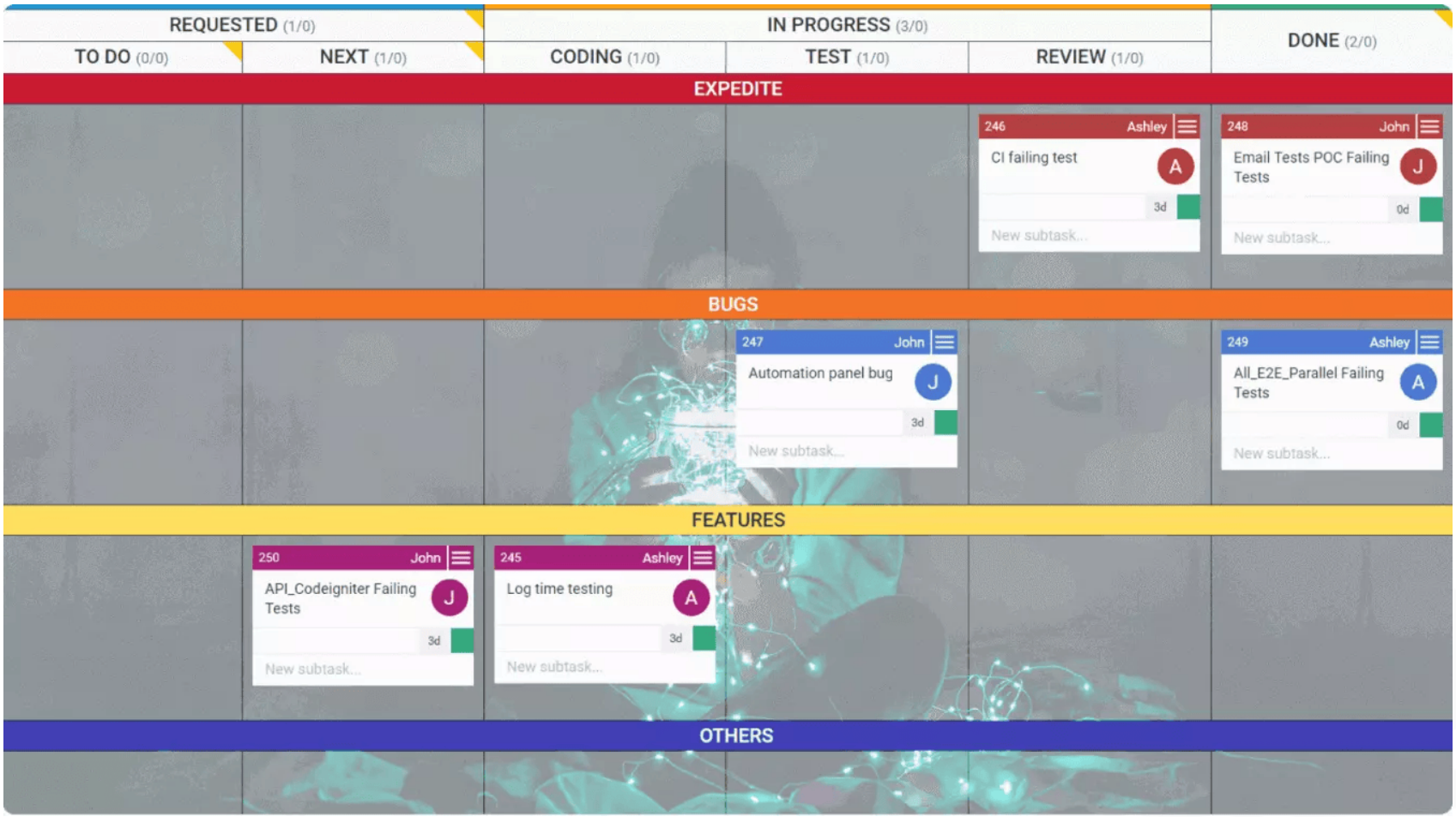
Tableros Kanban para equipos de QA



Tableros Kanban para equipos de QA

- Los **QA engineers** son responsables de mantener altos estándares en cada característica o producto.
- Dependiendo del escenario, para separar distintos tipos de trabajo, se pueden usar **swimlanes** como:
  - Bugs
  - Nuevas características
  - Expedite (prioridad urgente)
- Esto proporciona una visión clara del trabajo en progreso.

Tableros Kanban para equipos de QA



Gestiona el flujo

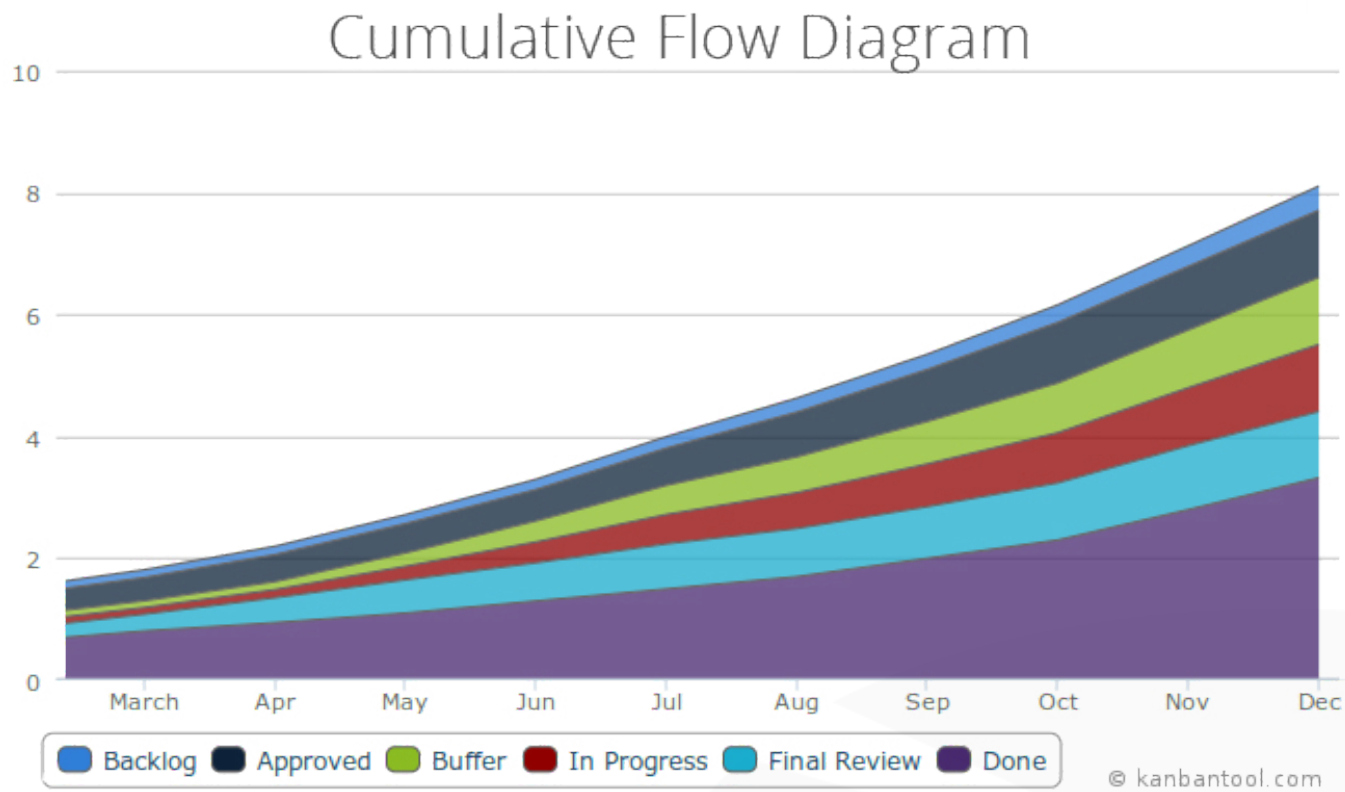
Gestiona el flujo: Diagrama de Flujo Acumulativo (CFD)

- El **Diagrama de Flujo Acumulativo** (CFD) es una de las herramientas analíticas más avanzadas en Kanban y métodos ágiles.
- El objetivo principal del **CFD** es mostrar cuán estable es tu flujo y ayudar a identificar áreas donde se necesita enfocar esfuerzos para hacer el proceso más predecible.

CFD: las tres métricas que muestra

- **Tiempo de Ciclo (Cycle Time)**: Comienza cuando un elemento de trabajo entra en el área activa del proceso y alguien empieza a trabajar en él.
- **Throughput**: Mide el número de elementos de trabajo que pasan por el sistema en un período específico.
- **Trabajo en Progreso (WIP)**: Indica el número de elementos en cada parte del sistema que están en curso y muestra la capacidad del equipo.

Gestiona el flujo: Diagrama de Flujo Acumulativo (CFD)



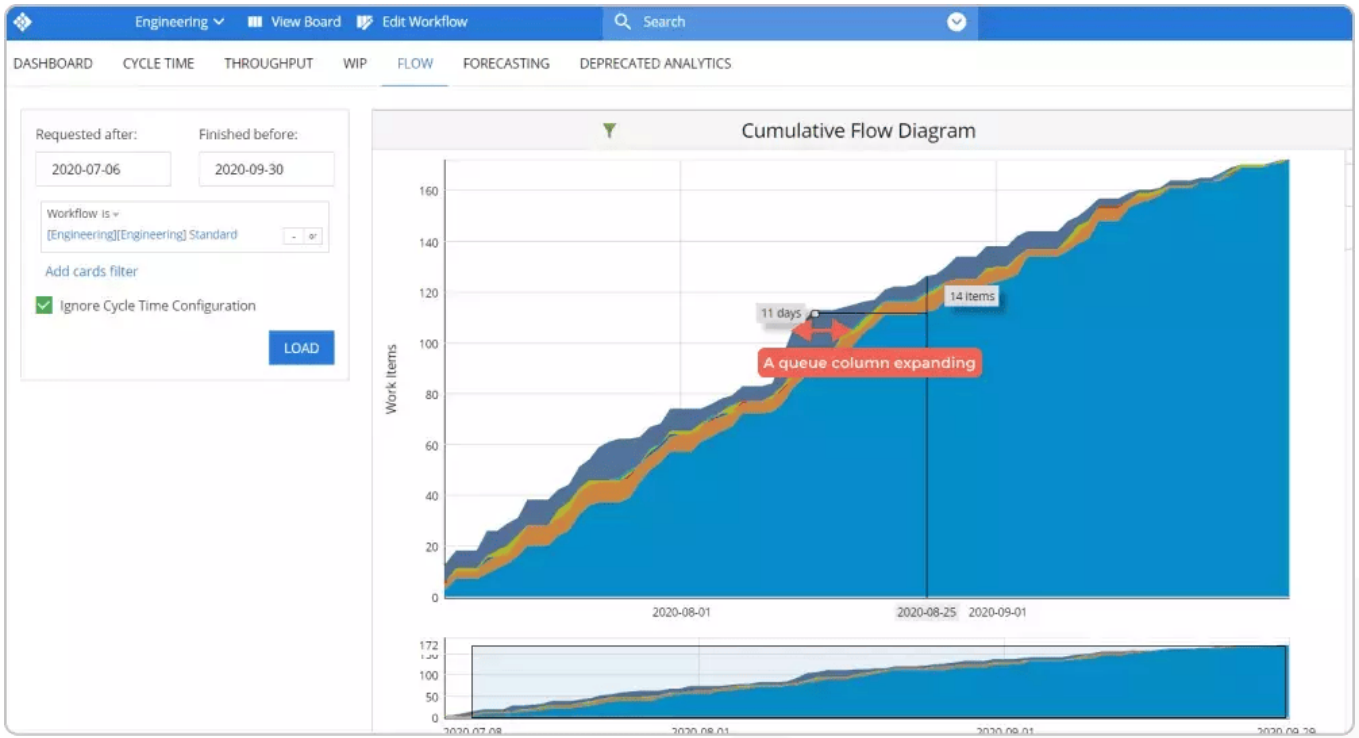
Cumulative Flow Diagram: Ejes y Secciones

- El eje **horizontal** del CFD representa el **marco temporal** durante el cual se visualizan los datos.
- El eje **vertical** muestra el **número acumulado de tarjetas** en el flujo de trabajo en diferentes momentos.
- Las bandas de colores que dividen las secciones del flujo representan las **etapas** del proceso de flujo de trabajo tal como aparecen en el tablero Kanban.

Análisis del CFD

- Si una línea en el CFD se vuelve **plana**, significa que no hay tareas entrando o saliendo de la etapa correspondiente.
- El **tiempo de ciclo aproximado** se puede estimar midiendo la distancia horizontal entre la línea superior de la primera etapa y la línea inferior de la última etapa "en progreso".
- Esta distancia refleja el tiempo que ha transcurrido (días, semanas, meses) y proporciona el **tiempo de ciclo promedio** de las tareas del equipo durante el periodo analizado.

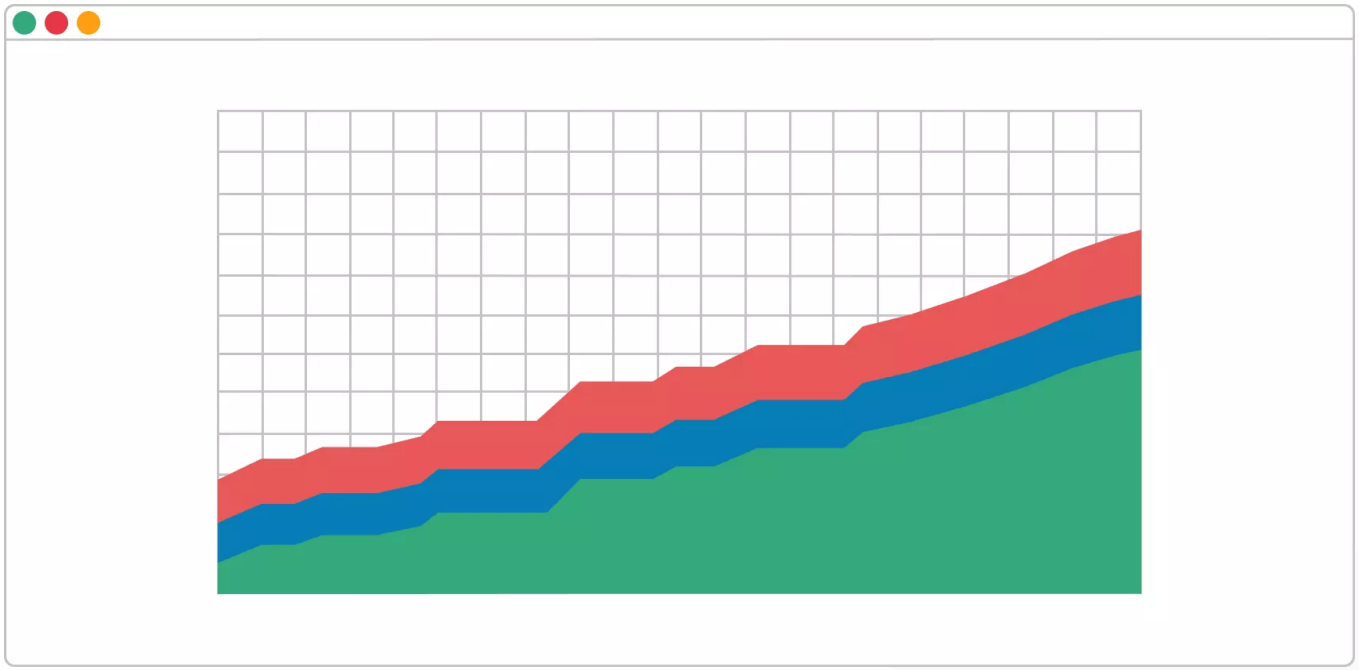
Gestiona el flujo: Diagrama de Flujo Acumulativo (CFD)



Ejemplo de CFD: Bandas en Paralelo

- **Interpretación**: Las bandas del diagrama avanzan de forma paralela.
- **Significado**: El rendimiento del flujo de trabajo es estable; las tareas entran y salen del flujo al mismo ritmo.
- **Acción**: Este es el escenario ideal. El equipo puede enfocarse en reducir los tiempos de ciclo.

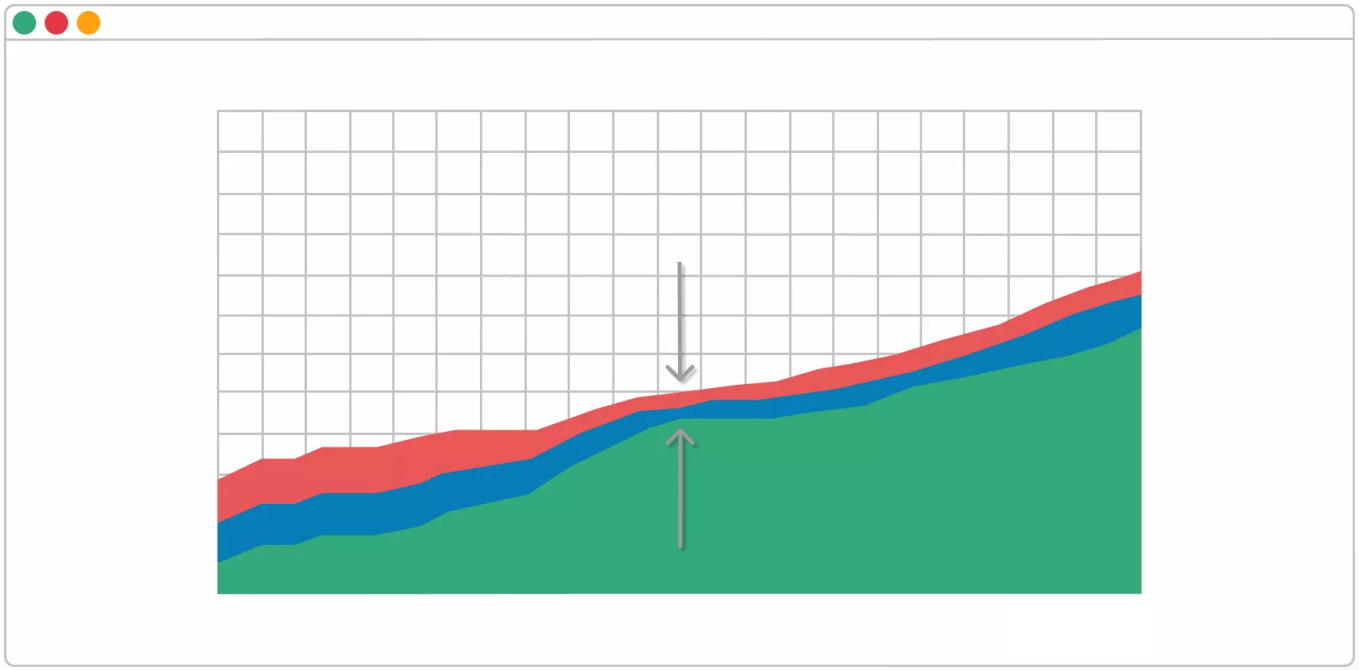
Gestiona el flujo: Diagrama de Flujo Acumulativo (CFD)



Ejemplo de CFD: Banda que se Estrecha Rápidamente

- **Interpretación**: Una banda en el diagrama se va estrechando progresivamente.
- El rendimiento en esa etapa es mayor que la tasa de entrada de tareas.
- **Acción**: Redistribuir la capacidad para optimizar el flujo y evitar exceso de recursos. Es decir, hay que ajustar la asignación de capacidad para equilibrar mejor la carga de trabajo.

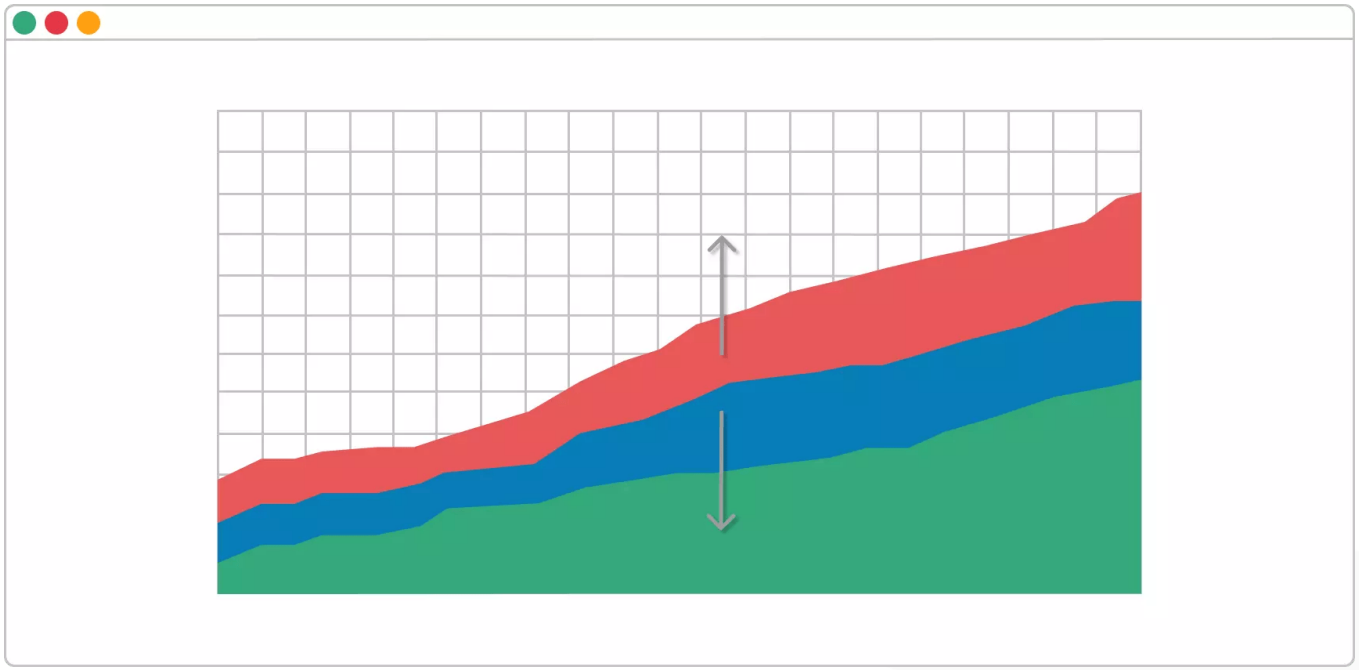
Gestiona el flujo: Diagrama de Flujo Acumulativo (CFD)



Ejemplo de CFD: Banda que se Ensancha Rápidamente

- **Interpretación**: Una banda en el diagrama se va ensanchando progresivamente.
- La cantidad de tareas que entran en esa etapa es mayor que las que salen, lo que genera un cuello de botella.
- **Acción**: Reconsiderar los límites de WIP y enfocarse en completar tareas en curso antes de iniciar nuevas. Un objetivo sería evitar la multitarea y otras actividades que no agregan valor al flujo de trabajo.

Gestiona el flujo: Diagrama de Flujo Acumulativo (CFD)



¿Qué pasa si descienden?

- Si una banda en el CFD baja, el diagrama es incorrecto, ya que las tareas no deben desaparecer del flujo.

Explicita las Políticas

Core Practices: Explicita las Políticas

- Definir políticas explícitas ayuda a alinear a los equipos y a garantizar que el trabajo fluya de manera consistente y predecible.
- Las políticas deben ser **claras**, **visibles** y entendidas por todo el equipo.
- Ejemplos de políticas importantes en Kanban:
  - **Definition of Done:** Criterios específicos que indican cuándo una tarea o historia está completamente terminada.
  - **Classes of Services:** Priorización según el tipo de trabajo, como urgencia o impacto.
  - **Service Level Agreements (SLA):** Acuerdos sobre tiempos esperados para completar diferentes tipos de trabajo.

Definition of Done

- Define qué significa que una tarea esté completamente terminada.
- Ejemplo de criterios:
  - Código revisado y aprobado.
  - Pruebas unitarias realizadas y aprobadas.
  - Documentación actualizada.
  - Historia de usuario aceptada por el Product Owner.
- Ayuda a evitar la ambigüedad y a mantener un estándar de calidad en el equipo.

Classes of Services

- Categorización de tareas basada en:
  - **Urgencia:** Tareas de tipo "Expedite" que requieren atención inmediata.
  - **Valor para el cliente:** Priorizar características o funcionalidades críticas.
  - **Deuda técnica:** Trabajo necesario para mantener la calidad del software.
- Cada clase de servicio puede tener políticas específicas:
  - Límites de WIP exclusivos.
  - Reglas de priorización especiales.
- Ejemplo: Las tareas "Expedite" siempre tienen prioridad sobre otras y se mueven en una "fast lane".

Service Level Agreements (SLA)

- Acuerdos que establecen tiempos máximos para completar diferentes tipos de trabajo.
- Ejemplo de SLA:
  - Las tareas de "Expedite" deben completarse en menos de 24 horas.
  - Las historias de usuario estándar deben completarse en un máximo de 5 días.
- Beneficios de los SLA:
  - Mejora la previsibilidad del proceso.
  - Establece expectativas claras con los stakeholders.
- Los SLA pueden visualizarse en el tablero Kanban con indicadores o métricas.

Introduce la Retroalimentación y Mejora

Core Practices: Retroalimentación y Mejora

- La retroalimentación y mejora continua son fundamentales para Kanban y el desarrollo ágil.
- Ayudan a identificar problemas, ajustar procesos y optimizar el flujo de trabajo.
- Se enfocan en:
  - Establecer un **ritmo y cadencia** para entregas predecibles.
  - Realizar reuniones como las ***Day standups***.
  - Reflexionar sobre los procesos a través de **retrospectivas**.

Retroalimentación y Mejora: Ritmo y Cadencia

> *"A regular cadence, or 'heartbeat,' establishes the capability of a team to reliably deliver working software at a dependable velocity. An organization that delivers at a regular cadence has established its process capability and can easily measure its capacity."*
>
> *Mary and Tom Poppendieck*

- Un ritmo regular (o "latido") es clave para garantizar la consistencia en la entrega.
- Beneficios:
  - Medir la capacidad del equipo de manera precisa.
  - Establecer confianza en las entregas regulares.

Retroalimentación y Mejora: Day Standups

- Las ***Day standups*** son reuniones diarias breves para revisar el flujo de trabajo.
- Cambian las preguntas típicas de Scrum para enfocarse en el flujo de Kanban:
  - ¿Qué nos está entorpeciendo?
  - ¿Cómo está ejecutándose el flujo?
  - ¿Qué podemos mejorar?
- Objetivo: Identificar bloqueos, evaluar el progreso y buscar mejoras inmediatas.

Retroalimentación y Mejora: Retrospectivas

- Espacios de reflexión para identificar:
  - Qué salió bien.
  - Qué podría mejorarse.
  - Acciones concretas para el próximo ciclo.
- Beneficios:
  - Promueven la mejora continua del equipo y los procesos.
  - Fomentan un ambiente de colaboración y aprendizaje.
- Herramientas comunes:
  - Técnicas como la estrella de mar (*start, stop, keep doing*).
  - Uso de métricas del flujo para informar las decisiones.

Conclusiones

Conclusiones

> *"Kanban is a gateway drug to lean"*
>
> *Chris Hefley*

- Kanban destaca por su capacidad de:
  - **Permitir entregas en cualquier momento**, favoreciendo la flexibilidad.
  - **Cambiar prioridades al vuelo**, adaptándose rápidamente a las necesidades del cliente.
  - **Visualizar perfectamente el flujo**, lo que permite identificar y resolver bloqueos.

Conclusiones: Beneficios Clave

- **Centrarse en la calidad:** Priorizar la entrega de valor real al cliente.
- **Reducir el Work-In-Progress:** Minimizar tareas simultáneas para mejorar la eficiencia.
- **Entregar con más frecuencia:** Incrementar el ritmo de entregas, favoreciendo ciclos de retroalimentación más rápidos.
- **Equilibrar la demanda con el rendimiento:** Asegurar que los equipos no están sobrecargados ni infrautilizados.
- **Priorizar de forma efectiva:** Alinear el trabajo con los objetivos estratégicos del negocio.
- **Atacar las fuentes de variabilidad:** Identificar y eliminar cuellos de botella y fuentes de ineficiencia para mejorar la predictibilidad.

Conclusiones: Reflexión Final

- Kanban no es solo una herramienta, es una filosofía que:
  - Promueve la mejora continua.
  - Ayuda a los equipos a ser más adaptables y efectivos.
  - Crea un puente hacia la adopción de prácticas Lean más avanzadas.
- Al implementar Kanban, las organizaciones pueden transformar su forma de trabajar, entregando mayor valor con menos esfuerzo.

End of Kanban